Webbasiertes Glossar

Prüfungsart: Fallstudie

Modul: DLBECGP01

Studiengang: Softwareentwicklung

29.03.2026 - Christopher Oelschläger - 102304401
Tutor: Alexander Christopher Bock

GitHub-Repository:

https://github.com/chrisoel/DLBECGP01

Inhaltsverzeichnis ..... **II**

Abbildungs- und Tabellenverzeichnis ..... **IV**

Abkürzungsverzeichnis ..... **V**

1. Einleitung **1**

1.1. Ausgangssituation 1

1.2. Zielsetzung 1

1.3. Annahmen & Abgrenzung 1

2. Anforderungen **2**

2.1. Funktionale Anforderungen (FA) 2

2.2. Qualitätsanforderungen (QA) 2

2.3. Randbedingungen (RB) 3

3. Begründete Entscheidung **4**

3.1. Mögliche Ansätze 4

3.2. Vergleich und Bewertung 5

3.3. Entscheidung für den Lösungsansatz 5

4. Konzeption / Entwurf **6**

4.1. Seitenstruktur 6

4.2. UI/UX-Konzept 6

4.3. Datenmodell 6

5. Umsetzung / Implementierung **7**

5.1. Technologie-Stack und Architektur 7

5.2. Implementierung der Geschäftslogik 7

5.3. Datenverarbeitung und -weitergabe 7

5.4. Dokumentation der Umsetzung 7

5.5. Dateistruktur und Modulübersicht 7

6. Test & Validierung **9**

6.1. Testansatz 9

6.2. Durchführung und Ergebnisse 9

6.3. Validierung qualitativer Merkmale 9

6.4. Dokumentation der Validierung 9

7. Ergebnis & Fazit **10**

Literaturverzeichnis **11**

Anhang 1 - Funktionale Anforderungen 12

Anhang 2 - Qualitätsanforderungen 16

Anhang 3 - Randbedingungen 18
Anhang 4 - Abbildung index.html 21
Anhang 5 - Abbildung Such-Funktion 22
Anhang 6 - Abbildung Zeitstempel-Funktion 23
Anhang 7 - Abbildung Begriffsseite 24
Anhang 8 - Abbildung Begriffseite-Navigation 25
Anhang 9 - Abbildung Verlinkung 26
Anhang 10 - index.html 27
Anhang 11 - testfall.html 29
Anhang 12 - testbedingung.html 31
Anhang 13 - testbasis.html 33
Anhang 14 - testabdeckung.html 35
Anhang 15 - aequivalenzklassenbildung.html 37
Anhang 16 - _vorlage.html 39
Anhang 17 - datenschutz.html 41
Anhang 18 - impressum.html 42
Anhang 19 - style.css 43
Anhang 20 - suche.js 53
Anhang 21 - letzteSpeicherung.js 55
Anhang 22 - footer.js 56
Anhang 23 - begriffeDaten.js 57
Anhang 24 - logo.js 61
Anhang 25 - startseiteGlossar.js 62
Anhang 26 - vorlesen.js 65
Anhang 27 - zurueckLink.js 67
Anhang 28 - begriffe.json 68
Anhang 29 - Wartungsanleitung (Auszug) 69

Abbildungs- und Tabellenverzeichnis

Tabelle 1 - FA-01 12
Tabelle 2 - FA-02 12
Tabelle 3 - FA-03 12
Tabelle 4 - FA-04 13
Tabelle 5 - FA-05 13
Tabelle 6 - FA-06 13
Tabelle 7 - FA-07 14
Tabelle 8 - FA-08 14
Tabelle 9 - FA-09 14
Tabelle 10 - FA-10 14
Tabelle 11 - FA-11 15
Tabelle 12 - QA-01 16
Tabelle 13 - QA-02 16
Tabelle 14 - QA-03 16
Tabelle 15 - QA-04 17
Tabelle 16 - QA-05 17
Tabelle 17 - QA-06 17
Tabelle 18 - RB-01 18
Tabelle 19 - RB-02 18
Tabelle 20 - RB-03 18
Tabelle 21 - RB-04 18
Tabelle 22 - RB-05 19
Tabelle 23 - RB-06 19
Tabelle 24 - RB-07 19
Tabelle 25 - RB-08 19
Tabelle 26 - RB-09 20
Tabelle 27 - RB-10 20
Tabelle 28 - RB-11 20
Abbildung 1 - index.html 21
Abbildung 2 - Such-Funktion 22
Abbildung 3 - Zeitstempel-Funktion 23
Abbildung 4 - Begriffsseite 24
Abbildung 5 - Begriffseite-Navigation 25
Abbildung 6 - Verlinkung 26

Abkürzungsverzeichnis

FA = Funktionale Anforderungen

QA = Qualitätsanforderungen

RB = Randbedingungen

ISTQB = International Software Testing Qualifications Board

CMS = Content-Management-System

CSS = Cascading Style Sheets

HTML = HyperText Markup Language

ID = Identifier / Kennung

ISTQB = International Software Testing Qualifications Board

IT = Informationstechnologie

JSON = JavaScript Object Notation

RE = Requirements Engineering

UI = User Interface

UX = User Experience

WYSIWYG = What You See Is What You Get

CSS3 = Cascading Style Sheets Level 3

HTML5 = HyperText Markup Language 5

DLBECGP01 = Modulcode Grundlagen der Web-Programmierung

IREN01 = Modulcode Requirements Engineering

1. Einleitung

Dieses Kapitel beschreibt die Ausgangssituation, die Zielsetzung sowie die Annahmen und Abgrenzungen der Fallstudie.

1.1. Ausgangssituation

Im Rahmen des Kurses DLBECGP01 bestand die Möglichkeit zwischen drei Aufgabenstellungen zu wählen. Für die vorliegende Arbeit wurde die erste Aufgabenstellung „Glossar für die Log7-Consult GmbH“ gewählt (IU Internationale Hochschule, n. d., S. 2).

1.2. Zielsetzung

Ziel der Arbeit ist die Konzeption und prototypische Entwicklung eines webbasierten Glossars für die Log7-Consult GmbH. Die Anwendung soll ein einheitliches Verständnis zentraler Fachbegriffe aus Logistik, IT und/oder Marketing fördern und die interne Kommunikation verbessern (IU Internationale Hochschule, n. d., S. 2).

Der Nutzen für die Zielgruppe (die Mitarbeitenden) liegt in einem schnellen, strukturierten Zugriff auf verständliche und verlässliche Begriffserklärungen sowie in der Unterstützung der bereichsübergreifenden Zusammenarbeit (IU Internationale Hochschule, n. d., S. 2).

Zu den zentralen Funktionen gehören die Darstellung von Glossarbegriffen auf einzelnen Seiten, die Verlinkung verwandter Inhalte sowie eine Suchfunktion zur gezielten Navigation (IU Internationale Hochschule, n. d., S. 2).

Qualitätsanforderungen betreffen insbesondere eine benutzerfreundliche Gestaltung, klare Struktur, gute Lesbarkeit sowie eine einfache Erweiterbarkeit der Inhalte (IU Internationale Hochschule, n. d., S. 2).

1.3. Annahmen & Abgrenzung

Annahmen:

Die Mitarbeitenden der Log7-Consult GmbH sind befähigt, die Webseite eigenständig auf einem Webspace bereitzustellen.

Sie besitzen grundlegende Kenntnisse im Umgang mit Texteditoren und können Inhalte selbst mit Hilfe einer Anleitung bearbeiten.

Abgrenzung:

Es handelt sich um einen Prototypen.

Rechtliche Hinweise werden nur statisch dargestellt und nicht juristisch geprüft.

Die Gestaltung orientiert sich an üblichen UI-Best-Practices, ohne eine umfassende Usability- oder Barrierefreiheitsprüfung.

2. Anforderungen

Dieses Kapitel beschreibt die methodische Anforderungsermittlung für das Online-Glossar. Requirements Engineering (RE) agiert dabei als „kooperativer, iterativer, inkrementeller Prozess“ (IU Internationale Hochschule, 2023, S. 296), um basierend auf der vorliegenden Aufgabenstellung (IU Internationale Hochschule, n. d., S. 2) alle relevanten funktionalen Anforderungen (FA), Qualitätsanforderungen (QA) und Randbedingungen (RB) im nötigen Detailgrad zu klären. Ziel ist es zu gewährleisten, dass alle Anforderungen bekannt sind und im erforderlichen Maße verstanden werden (IU Internationale Hochschule, 2023, S. 296). Zur Sicherung der Überprüfbarkeit ist jede Anforderung mit einer eindeutigen ID sowie messbaren Akzeptanzkriterien versehen (IU Internationale Hochschule, 2023, S. 520, 552), welche in den Anhängen 1-3 detailliert aufgeführt sind.

2.1. Funktionale Anforderungen (FA)

Funktionale Anforderungen definieren die vom System bereitzustellenden fachlichen Funktionen (IU Internationale Hochschule, 2023, S. 310).

Der fachliche Umfang (*FA-02*) ist so festzulegen, dass das Glossar genau fünf Fachbegriffe umfasst, die vom Ersteller frei gewählt werden können. Diese Begriffe müssen jedoch eindeutig den Clustern Logistik, IT oder Marketing zugeordnet werden können und fachlich klar einem dieser Bereiche entstammen (IU Internationale Hochschule, n. d., S. 2). Für die Strukturierung der Web-Anwendung (*FA-03*) ist eine übersichtliche und hierarchische Gliederung umzusetzen. Diese besteht aus einer zentralen Startseite sowie fünf dazugehörigen Unterseiten. Daraus ergibt sich technisch die Erstellung von insgesamt mindestens sechs separaten HTML-Dateien (IU Internationale Hochschule, n. d., S. 2). Zur Verbesserung der Benutzerfreundlichkeit (*FA-10*) muss die Anwendung außerdem über eine clientseitige Suchfunktion verfügen. Diese ist mithilfe von JavaScript zu realisieren und soll es den Nutzer:innen ermöglichen, Suchbegriffe einzugeben und daraufhin passende Ergebnisse beziehungsweise direkte Verlinkungen zu den entsprechenden Begriffsseiten zu erhalten (IU Internationale Hochschule, n. d., S. 2).

Die vollständige Liste der funktionalen Anforderungen (*FA-01* bis *FA-11*) ist in *Anhang 1* dokumentiert.

2.2. Qualitätsanforderungen (QA)

Qualitätsanforderungen ergänzen funktionale Anforderungen um qualitative Eigenschaften (IU Internationale Hochschule, 2023, S. 311). Das Informationsdesign (*QA-02*) ist verständlich, konsistent und für ein Glossar geeignet umzusetzen. Dazu gehört eine einheitliche Gestaltung von Navigation, Seitenstruktur und Inhaltsanordnung, sodass Nutzer:innen intuitiv zwischen den Begriffsseiten navigieren können (IU Internationale Hochschule, n. d., S. 2). Die Suchfunktion (*QA-03*) muss zuverlässig und fehlerfrei arbeiten. Sie ist so zu gestalten, dass relevante Begriffe bei typischen Eingaben (z. B. exakte Treffer oder Präfix-Suche) korrekt erkannt und den Nutzer:innen angezeigt werden (IU Internationale Hochschule, n. d., S. 2).

Weitere Aspekte sind in *Anhang 2* detailliert aufgeführt.

2.3. Randbedingungen (RB)

Randbedingungen definieren organisatorische oder technische Systemvorgaben (IU Internationale Hochschule, 2023, S. 312). Die technische Umsetzung (*RB-03*) ist auf die Web-Kerntechnologien HTML, CSS und JavaScript zu beschränken (IU Internationale Hochschule, n. d., S. 2). Der Quellcode (*RB-05*, *RB-06*) muss vollständig eigenständig in einem Editor erstellt werden. Der Einsatz von Content-Management-Systemen wie WordPress oder grafischen WYSIWYG-Editoren, die Code automatisch generieren (z.B. durch grafisches Einfügen von Elementen), ist ausdrücklich untersagt (IU Internationale Hochschule, n. d., S. 3).

Eine vollständige Übersicht aller Randbedingungen findet sich in *Anhang 3*.

3. Begründete Entscheidung

In diesem Kapitel werden mögliche Lösungsansätze für den Glossar-Prototyp gegenübergestellt und bewertet. Ziel ist es, nachvollziehbar zu begründen, warum der gewählte Ansatz die fachlichen Anforderungen, Qualitätsanforderungen und Randbedingungen der Fallstudie am besten erfüllt.

3.1. Mögliche Ansätze

Ansatz 1: Rein statische HTML-Einzelseiten ohne zentrale Datenhaltung

Bei diesem Ansatz werden Startseite, Suche und alle Begriffsseiten komplett manuell gepflegt. Begriffsname, Link und Kurzbeschreibung sind direkt in den einzelnen HTML-Dateien enthalten, während die Suchdaten fest im JavaScript hinterlegt werden. Es gibt keine zentrale Datenquelle für die Begriffsmetadaten.

Dieser Ansatz bietet einen sehr einfachen technischen Einstieg, benötigt keine zusätzliche Datenstruktur und eignet sich gut für kleine, einmalige Prototypen. Nachteile sind die hohe Doppelpflege bei neuen oder geänderten Begriffen, die größere Fehleranfälligkeit durch manuelle Anpassungen an mehreren Stellen und die geringe Wartbarkeit bei Erweiterungen.

Insgesamt ist der initiale Aufwand gering, steigt jedoch mit der Zeit an. Die Komplexität bleibt niedrig, die Wartbarkeit jedoch niedrig.

Ansatz 2: Dynamische Webanwendung mit Backend oder CMS

Bei diesem Ansatz werden Begriffe über ein Backend, eine Datenbank oder ein Content-Management-System verwaltet und dynamisch ausgeliefert. Inhalte, Logik und Darstellung sind technisch getrennt, und eine zentrale Verwaltungsoberfläche ermöglicht komfortable Pflege.

Dieser Ansatz bietet sehr gute Skalierbarkeit, einfache Erweiterbarkeit bei vielen Inhalten sowie eine solide Grundlage für Rollen, Freigaben und Mehrbenutzerbetrieb. Nachteile sind der deutlich höhere Implementierungsaufwand, zusätzlicher Betriebs- und Pflegeaufwand und die Überdimensionierung für einen einfachen Prototyp.

Insgesamt ist der Aufwand hoch, die Komplexität hoch, und die Wartbarkeit im Zielbild sehr gut, jedoch mit erheblichem initialem Aufwand verbunden.

Ansatz 3: Statischer Mehrseitenansatz mit zentraler JSON-Datei und HTML-Vorlage

Bei diesem Ansatz bleiben die Glossarseiten statische HTML-Dateien, während Begriffsname, Link und Kurzbeschreibung zentral in einer JSON-Datei gepflegt werden. Startseite und Suche laden die Daten per JavaScript, und neue Seiten werden auf Basis einer Vorlage erstellt. Gemeinsame Funktionen wie Suche, Footer, Logo und Zeitstempel werden ebenfalls zentral bereitgestellt.

Dieser Ansatz bietet eine gute Balance zwischen Einfachheit und Wartbarkeit: Doppelpflege wird reduziert, neue Begriffe lassen sich mit geringem technischem Aufwand hinzufügen, und ein Backend ist nicht erforderlich. Nachteile sind, dass die Inhalte der Unterseiten weiterhin manuell gepflegt werden müssen, bei stark wachsendem Umfang zusätzliche Professionalisierung nötig ist und das Laden der JSON-Datei einen Webserver voraussetzt.

Insgesamt ist der Aufwand mittel, die Komplexität mittel und die Wartbarkeit gut.

3.2. Vergleich und Bewertung

Ansatz 1 ist nur begrenzt skalierbar, da neue Begriffe an mehreren Stellen manuell ergänzt werden müssen. Ansatz 2 bietet die beste technische Skalierbarkeit, ist für einen Prototyp mit nur fünf Begriffen jedoch überdimensioniert. Ansatz 3 hingegen skaliert für den vorgesehenen Umfang gut, da neue Begriffe strukturiert über die Vorlage und die JSON-Datei hinzugefügt werden können. Zudem ist die Fehleranfälligkeit geringer als bei einer rein statischen Lösung, da Suchdaten und Startseiteninformationen nicht mehrfach gepflegt werden müssen.

3.3. Entscheidung für den Lösungsansatz

Gewählt wurde der statische Mehrseitenansatz mit zentraler JSON-Datei für die Begriffsdaten, einer HTML-Vorlage für neue Begriffsseiten und modularen JavaScript-Dateien für gemeinsame Funktionen.

Dieser Ansatz passt gut zu den Anforderungen des Prototyps, da er ohne unnötigen technischen Overhead auskommt und die Pflichttechnologien HTML, CSS und JavaScript vollständig umsetzt. Er bietet eine bessere Wartbarkeit als eine rein statische Lösung, ermöglicht einfache Erweiterungen für neue Begriffe über Vorlage und JSON-Pflege und sorgt durch eine einheitliche Seitenstruktur für eine klare und konsistente Benutzerführung.

4. Konzeption / Entwurf

Das Konzept setzt die Anforderungen in eine übersichtliche, benutzerfreundliche und wartbare Architektur um. Dazu wurde eine Multi-Page-Application mit HTML, CSS und JavaScript umgesetzt, wobei die Begriffsseiten eigenständig sind und gemeinsame Metadaten zentral in einer JSON-Datei verwaltet werden.

4.1. Seitenstruktur

Die Anwendung besteht aus einer Startseite und fünf Glossarbegriffsseiten. Gemeinsame Elemente wie Header, Footer und Navigation werden zentral über JavaScript erzeugt, um Konsistenz zu gewährleisten.

- Header: Logo mit Link zur Startseite, Suchfunktion mit Live-Trefferliste. Auf Begriffsseiten zusätzlicher Titelbereich mit Zurück-Pfeil, Home-Button, Hauptüberschrift und Vorlesefunktion.
- Hauptbereich: Startseite zeigt Einführung, A-Z-Filter und tabellarische Übersicht. Begriffsseiten enthalten Definition, weiterführende Informationen, interne und externe Links.
- Footer: Zentrale Erzeugung mit Links zu Startseite, Datenschutz, Impressum, Datum/Uhrzeit der letzten Speicherung und Copyright-Hinweis.

Der Navigationsfluss ist einfach und intuitiv: Nutzer:innen gelangen über Tabelle, Filter oder Suche zu den Begriffsseiten und können von dort über interne Links, Zurück- oder Home-Buttons navigieren.

4.2. UI/UX-Konzept

Die Oberfläche ist klar strukturiert, konsistent gestaltet und responsiv für Desktop, Tablet und Smartphone, mit wiedererkennbaren Elementen wie Überschriften, Hervorhebungen, Tabellen und Symbol-Buttons. Die Farbgebung in Blau, Weiß und Gelb unterstützt das Branding. Die Benutzerführung setzt auf schnelle Orientierung, direkte Suche, A-Z-Filter und interne Verlinkungen. Grundlegende Barrierefreiheitsaspekte sind umgesetzt, und Validierung sowie Feedback gewährleisten die direkte Anzeige von Eingaben und Fehlern, etwa bei leeren Filtern, fehlenden Glossardaten oder ungültigen JSON-Einträgen.

4.3. Datenmodell

Das Datenmodell basiert auf einer zentralen JSON-Datei, die Metadaten aller Glossareinträge enthält. Jeder Glossareintrag (name, link, beschreibung) verweist auf eine eigenständige Begriffsseite, die Hauptüberschrift, Definition, weiterführende Informationen und Links enthält.

Abgeleitete Daten wie Gruppenzuordnung, Sortierung nach deutscher Sprachlogik und Generierung von Gruppen-IDs für die A-Z-Navigation werden zur Laufzeit erstellt. Das Modell reduziert den Pflegeaufwand erheblich: Neue Begriffe werden durch Anlegen einer HTML-Seite aus der Vorlage und Ergänzen des JSON-Eintrags hinzugefügt. Dadurch bleibt die Erweiterung nachvollziehbar und wartbar, selbst für weniger erfahrene Mitarbeitende.

5. Umsetzung / Implementierung

Die Implementierung überführt die Konzeption in einen funktionsfähigen Glossar-Prototyp. Fokus liegt auf verständlicher Struktur, wartbarer Umsetzung und nachvollziehbaren technischen Entscheidungen.

5.1. Technologie-Stack und Architektur

Der Prototyp verwendet einfache, weit verbreitete Webtechnologien: HTML5 für Struktur, CSS3 für Layout und Styling sowie reines JavaScript für Suche, Navigation, Vorlesefunktion und Footer. Die Glossardaten liegen zentral in einer JSON-Datei. Frameworks, Build-Tools oder Backend-Systeme kommen nicht zum Einsatz. Die Multi-Page-Application besteht aus einer Startseite, fünf Glossarbegriffsseiten sowie ergänzenden Seiten für Datenschutz und Impressum. Wiederkehrende Funktionen wurden in modulare JavaScript-Dateien ausgelagert, um Wartbarkeit, klare Verantwortlichkeiten und einfache Erweiterbarkeit zu gewährleisten.

5.2. Implementierung der Geschäftslogik

Die zentralen Glossardaten werden beim Laden über fetch() eingelesen, validiert und für Suche, Filterleiste und Tabellenansicht bereitgestellt. Der interne Zustand (z. B. Suchtext, Trefferliste, Buchstabenfilter, Ladezustand) wird dynamisch im Browser verwaltet, wodurch die Benutzeroberfläche kontextbezogen reagiert. Validierung und Fehlerbehandlung sorgen dafür, dass nur gültige Einträge verarbeitet werden, Suchtreffer live angezeigt werden und Nutzer:innen klare Rückmeldungen bei Problemen erhalten.

5.3. Datenverarbeitung und -weitergabe

Alle Datenverarbeitungen erfolgen clientseitig. Es gibt kein Backend, keine persistente Speicherung und keinen Mehrbenutzerbetrieb. Dies reicht für die Demonstration der Kernfunktionen des Prototyps vollständig aus.

5.4. Dokumentation der Umsetzung

Die Umsetzung ist durch Kommentare, Dokumentenköpfe, eine Pflegeanleitung sowie Kapitel zur Entscheidungsbegründung nachvollziehbar dokumentiert. Visuelle Nachweise wie Startseite, Begriffsseiten, Footer und responsives Verhalten können durch Screenshots oder Mockups ergänzt werden. Funktionale Prüfungen wurden manuell durchgeführt, insbesondere Suchfunktion, Navigation, Filterlogik, Footer-Zeitstempel, Konsistenz und Responsivität. Nicht-funktionale Anforderungen wie Benutzerfreundlichkeit, Wartbarkeit, Konsistenz, Responsivität und Robustheit wurden berücksichtigt.

5.5. Dateistruktur und Modulübersicht

Die technische Umsetzung des Glossar-Prototyps ist im src-Ordner modular strukturiert, um Inhalte, Darstellung und Logik klar zu trennen und die Wartbarkeit zu erhöhen.

Im Wurzelverzeichnis von src befindet sich die zentrale Startseite **index.html** (siehe *Anhang 10*), die Einleitung, Platzhalter für Topbar und Footer sowie Zielcontainer für Filterleiste und tabellari-

sche Glossarübersicht enthält. Ergänzt werden die Platzhalterseiten **datenschutz.html** (siehe *Anhang 17*) und **impressum.html** (siehe *Anhang 18*), die die gemeinsame Navigations- und Footerstruktur nutzen. Das Stylesheet **style.css** (siehe *Anhang 19*) bündelt Layout, Farb- und Abstandsvariablen, responsive Anpassungen sowie die Gestaltung von Suchfeld, Trefferlisten, Filtern, Tabellen, Buttons und Footer. logo.png dient als Branding-Element, favicon.svg als Browser-Icon. Die Datei **begriffe.json** (siehe *Anhang 28*) bildet die zentrale Datenbasis mit Metadaten aller Glossarbegriffe (name, link, beschreibung) und wird von Startseite und Suche gemeinsam genutzt.

Im Unterordner src/begriffe befinden sich die einzelnen Glossarseiten. **_vorlage.html** (siehe *Anhang 16*) dient als standardisierte Vorlage für neue Begriffsseiten, während die Seiten wie **testfall.html** (siehe *Anhang 11*), **testbedingung.html** (siehe *Anhang 12*), **testbasis.html** (siehe *Anhang 13*), **testabdeckung.html** (siehe *Anhang 14*) und **aequivalenzklassenbildung.html** (siehe *Anhang 15*) beispielhaft Begriffe erklären und fachliche Verknüpfungen darstellen. Alle Glossarseiten folgen einer einheitlichen Struktur aus Topbar, Seitenkopf mit Navigation, Definitionsbereich, weiterführenden Informationen und Footer, was Konsistenz und einfache Pflege gewährleistet.

Die Anwendungslogik im Unterordner src/js ist in klar abgegrenzte Module unterteilt:

- **begriffeDaten.js** (siehe *Anhang 23*) lädt, validiert, normalisiert und sortiert die Glossardaten.
- **startseiteGlossar.js** (siehe *Anhang 25*) rendert Filterleiste und tabellarische Übersicht, inklusive Buchstabengruppen, Mehrfachfilter und Vorlese-Buttons.
- **suche.js** (siehe *Anhang 20*) realisiert die Live-Suche und behandelt Ladefehler sichtbar.
- **logo.js** (siehe *Anhang 24*) erzeugt Topbar mit Logo und Suchfeld zentral.
- **footer.js** (siehe *Anhang 22*) erstellt den gemeinsamen Footer.
- **letzteSpeicherung.js** (siehe *Anhang 21*) ergänzt den Footer um den Zeitstempel der letzten Speicherung.
- **vorlesen.js** (siehe *Anhang 26*) kapselt die browserbasierte Vorlesefunktion und erzeugt Vorlese-Buttons.
- **zurueckLink.js** (siehe *Anhang 27*) implementiert das Rücksprungverhalten für den Zurück-Button.

Die modulare Struktur stellt sicher, dass Inhalte, Styling und Logik sauber getrennt, wiederkehrende Elemente zentralisiert und neue Begriffe unkompliziert ergänzt werden können. Dies unterstützt sowohl die Nachvollziehbarkeit der Implementierung als auch die langfristige Wartung.

6. Test & Validierung

Die Qualitätssicherung des Glossar-Prototyps erfolgte ausschließlich durch gezielte manuelle Tests. Aufgrund des begrenzten Funktionsumfangs wurde auf automatisierte Tests verzichtet. Stattdessen wurden szenariobasierte Prüfungen durchgeführt, um die korrekte Umsetzung der funktionalen Anforderungen sowie wesentlicher Qualitätsmerkmale sicherzustellen.

6.1. Testansatz

Die Tests wurden aus Sicht der Nutzer:innen durchgeführt und orientierten sich an typischen Nutzungsszenarien, darunter Startseitenaufruf, Suche, Navigation zu Begriffsseiten, Anwendung von Filtern auf der Startseite sowie interne Verlinkungen. Die Validierung erfolgte schrittweise anhand von Black-Box-Prüfungen, wodurch die sichtbare Funktionalität überprüft wurde, ohne auf die interne Implementierung zurückzugreifen und White-Box-Prüfungen für einen Abgleich der Datenkonsistenz.

6.2. Durchführung und Ergebnisse

Die Tests wurden in den Browsern Safari und Chrome unter Einsatz von Visual Studio Code durchgeführt. Es wurde überprüft, dass Begriffe korrekt aus der JSON-Datei übernommen, alphabetisch sortiert und konsistent auf der Startseite, in der Filterleiste sowie in den Suchergebnissen dargestellt werden. Die Navigation über interne Links sowie die Funktionen der Zurück- und Home-Buttons arbeiteten fehlerfrei. Filter- und Mehrfachauswahl reagierten erwartungsgemäß, Suchtreffer wurden live angezeigt, und Datum sowie Uhrzeit im Footer stimmten.

Alle definierten Prüfszenarien wurden erfolgreich durchlaufen und kritische Abweichungen traten nicht auf.

6.3. Validierung qualitativer Merkmale

Gebrauchstauglichkeit, Responsivität und grundlegende Barrierefreiheit wurden ebenfalls manuell überprüft. Die Benutzeroberfläche ist verständlich, die Navigation klar strukturiert, Filter- und Suchfunktionen intuitiv bedienbar, und Inhalte bleiben auch auf kleineren Displays gut lesbar. Fehlerfälle wie nicht ladbare Glossardaten oder ungültige JSON-Einträge werden stabil abgefangen und führen zu verständlichen Rückmeldungen für die Nutzer:innen.
Während der Sichtprüfung der Suchfunktion zeigte sich, dass die Begrenzung der Anzeige auf drei Suchergebnisse sowie eine Scrollbarkeit die Bedienbarkeit insbesondere bei größeren Begriffsmengen verbessern würde. Die Umsetzung wurde entsprechend angepasst.

6.4. Dokumentation der Validierung

Die Ergebnisse wurden primär durch Sichtprüfung und praktische Nutzung dokumentiert. Ergänzende visuelle Nachweise befinden sich in den jeweiligen Anhängen. Screenshots der Startseite (siehe *Anhang 4*), Filter, Begriffsseiten (siehe Beispiel *Anhang 7*) und Footer (*Anhang 6*) befinden sich in den Anhängen.

7. Ergebnis & Fazit

Der Glossar-Prototyp stellt eine funktionsfähige Webanwendung dar, die Startseite, fünf fachlich verknüpfte Begriffsseiten, Suchfunktion, einheitliche Navigation und Anzeige des letzten Speicherzeitpunkts bereitstellt. Manuelle Validierung bestätigte, dass die zentralen Funktionen zuverlässig im vorgesehenen Nutzungskontext arbeiten.

Während der Umsetzung wurden einige Anpassungen vorgenommen, um Verständlichkeit, Wartbarkeit und technische Angemessenheit zu verbessern: Die Begriffsmetadaten wurden in eine zentrale JSON-Datei ausgelagert, eine HTML-Vorlage für neue Seiten eingeführt und wiederkehrende Elemente wie Topbar und Footer in JavaScript-Module überführt. Auf Backend, Frameworks und umfangreiche Testautomatisierung wurde bewusst verzichtet, um den Prototyp schlank und nachvollziehbar zu halten.

Die Traceability zwischen Anforderungen, technischer Umsetzung und Validierung ist durch modulare Dateien, klare Verantwortlichkeiten, Kommentare und Dokumentation gewährleistet.

Die eingesetzten Technologien HTML, CSS, JavaScript und JSON erwiesen sich als geeignet: Sie ermöglichten geringen Entwicklungsaufwand, schnelle Performance, gute Erweiterbarkeit, klare Benutzerführung und hohe Transparenz der technischen Struktur.

Fazit und Ausblick: Das Projektziel, einen funktionsfähigen Glossar-Prototyp für die Log7-Consult GmbH zu erstellen, wurde erfolgreich erreicht. Die Anwendung erfüllt die analysierten und dokumentierten Anforderungen vollständig.

Im Rahmen der Implementierung zeigte sich als zentrale Erkenntnis ein Missverständnis hinsichtlich der Anforderungen an die Wartbarkeit durch das Personal. Dies führte zu einem unnötigen Mehraufwand bei der Implementierung einer Funktion zur Hinzufügung neuer Begriffe. Für zukünftige Projekte lässt sich daraus ableiten, dass eine noch gründlichere Analyse der Aufgabenstellung und Anforderungen von entscheidender Bedeutung ist.

Für die Weiterentwicklung des Prototyps bieten sich sowohl funktionale als auch technische Erweiterungen an. Funktional könnten zusätzliche Begriffe, Kategorien oder Synonyme implementiert werden. Technisch sind ein Ausbau um Backend-Anbindungen, Schnittstellen zu anderen Systemen sowie Optimierungen in den Bereichen Barrierefreiheit, Sicherheit und redaktionelle Pflege denkbar. Mittelfristig könnten Automatisierung von Tests, Datenvalidierungen und Veröffentlichungsprozessen die Skalierbarkeit und den Pflegekomfort der Anwendung deutlich verbessern.

Literaturverzeichnis

- IU Internationale Hochschule. (n. d.). Fallstudie: DLBECGP01 – Grundlagen der Web-Programmierung . Prüfungsamt.
- IU Internationale Hochschule. (2023). Requirements Engineering (IREN01) (Versionsnr. 003-2023-0817). IU Internationale Hochschule GmbH.
- IU Internationale Hochschule. (2025). Grundlagen der Web-Programmierung (DLBECGP01) (Versionsnr. 001-2025-0124). IU Internationale Hochschule GmbH.

Anhang 1 - Funktionale Anforderungen

ID	FA-01
Quelle	*„Erstelle einen Prototyp für das Glossar [...]“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die Lösung muss als funktionsfähiger Prototyp eines Online-Glossars umgesetzt werden.
Akzeptanzkriterium	Ein im Browser ausführbarer Prototyp liegt vor.
Nachweis	Der Prototyp kann über das Live-Server-Plugin von Visual Studio Code ausgeführt werden.

Tabelle 1 - FA-01

ID	FA-02
Quelle	*„[...] für fünf selbst gewählte Begriffe, wahlweise aus den Bereichen Logistik, IT oder Marketing.“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Das Glossar muss genau fünf Fachbegriffe aus den vorgegebenen Clustern (Logistik, IT, Marketing) enthalten.
Akzeptanzkriterium	Es sind fünf Begriffe dokumentiert. Jeder ist fachlich einem der drei Bereiche zuordenbar.
Nachweis	Es wurden die folgenden fünf Begriffe gewählt: Testfall (siehe *Anhang 11*), Testbedingung (siehe *Anhang 12*), Testbasis (siehe *Anhang 13*), Testabdeckung (siehe *Anhang 14*) und Äquivalenzklassenbildung (siehe *Anhang 15*).

Tabelle 2 - FA-02

ID	FA-03
Quelle	*„Das Glossar soll aus einer Startseite sowie aus fünf Unterseiten bestehen [...]“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die Anwendung muss eine klare hierarchische Struktur aus Hauptseite und fünf Begriffsseiten aufweisen.
Akzeptanzkriterium	Es existieren insgesamt mindestens sechs HTML-Dateien: 1 Index-Seite und 5 Detail-Seiten.
Nachweis	Es existieren eine index.html (siehe *Anhang 10*), sowie testfall.html (siehe *Anhang 11*), testbedingung.html (siehe *Anhang 12*), testbasis.html (siehe *Anhang 13*), testabdeckung.html (siehe *Anhang 14*) und aequivalenzklassenbildung.html (siehe *Anhang 15*). Hinweis: Zusätzlich existiert eine _vorlage.html (siehe *Anhang 16*) für eine bessere Wartbarkeit. Sowie datenschutz.html (siehe *Anhang 17*) und impressum.html (siehe *Anhang 18*) gemäß rechtlicher Vorgaben.

Tabelle 3 - FA-03

ID	FA-04
Quelle	*„[...] auf denen jeweils ein Begriff erläutert wird.“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Jede Unterseite dient exklusiv der fachlichen Erläuterung eines einzelnen Begriffs.
Akzeptanzkriterium	Jede der fünf Unterseiten enthält eine individuelle Definition und Erläuterung.

ID	FA-04
Nachweis	Jede der fünf Unterseiten liefert die Definition und Erläuterung für genau ein Begriff: • testfall.html (siehe *Anhang 11*), • testbedingung.html (siehe *Anhang 12*), • testbasis.html (siehe *Anhang 13*), • testabdeckung.html (siehe *Anhang 14*) und • aequivalenzklassenbildung.html (siehe *Anhang 15*).

Tabelle 4 - FA-04

ID	FA-05
Quelle	*„Wähle die fünf Begriffe so, dass sinnvolle Verlinkungen zwischen den Begriffen und zu externen Seiten eingefügt werden." (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die Begriffe müssen intern vernetzt und mit externen Fachquellen verlinkt sein.
Akzeptanzkriterium	Jede Begriffsseite enthält mindestens einen funktionierenden internen Link zu einem anderen Glossarbegriff und einen externen Link.
Nachweis	Jede der fünf Unterseiten besitzt mindestens einen internen Link: • testfall.html (siehe *Anhang 11*) - Testfall zeigt auf Testbedingung und Äquivalenzklassenbildung, • testbedingung.html (siehe *Anhang 12*) - Testbedingung zeigt auf Testfall und Testbasis, • testbasis.html (siehe *Anhang 13*) - Testbasis zeigt auf Testbedingungen, • testabdeckung.html (siehe *Anhang 14*) - Testabdeckung zeigt auf Testbasis, Testfall und Testbedingung und • aequivalenzklassenbildung.html (siehe *Anhang 15*) - Äquivalenzklassenbildung zeigt auf Testfall und Testabdeckung. Sowie einen externen Link auf das ISTQB-Glossar (siehe *https://glossary.istqb.org/de_DE/search?term=&exact_matches_first=true*) und der passenden fachlichen Definition.

Tabelle 5 - FA-05

ID	FA-06
Quelle	*„Erstelle im Rahmen des Informationsdesigns zudem ein geeignetes Konzept für das Glossar." (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die Informationsarchitektur und Benutzerführung müssen konzeptionell geplant werden.
Akzeptanzkriterium	Die Arbeit enthält ein dokumentiertes Konzept zu Struktur und Navigation.
Nachweis	Das Konzept zu Struktur und Navigation befindet sich in *Kapitel 4 "Konzeption / Entwurf"*.

Tabelle 6 - FA-06

ID	FA-07
Quelle	*„Erstelle das Glossar in HTML." (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die technische Basis der Seiten muss die Auszeichnungssprache HTML sein.
Akzeptanzkriterium	Alle Seiten liegen als valide HTML-Dokumente vor.

ID	FA-07
Nachweis	Alle Seiten liegen als HTML-Dokumente vor. Zur Validierung der Dokumente wurde *https://validator.w3.org/* verwendet. Zum Zeitpunkt der Abgabe lag folgendes Ergebnis vor: „Document checking completed. No errors or warnings to show.“.

Tabelle 7 - FA-07

ID	FA-08
Quelle	*„Nutze für das Glossar verschiedene Gestaltungselemente wie beispielsweise Überschriften, Hervorhebungen, Tabellen, Grafiken [...]“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die Inhalte müssen durch diverse HTML-Strukturelemente semantisch aufbereitet werden.
Akzeptanzkriterium	Im Glossar sind Überschriften (<h1>-<h6>), Tabellen (<table>) und Grafiken (<img>) funktional eingebunden.
Nachweis	Die Anforderung wurde durch den Einsatz verschiedener Gestaltungselemente umgesetzt. Dazu zählen Überschriften zur Strukturierung, Hervorhebungen (z. B. farbige Boxen und Formatierungen), Tabellen zur übersichtlichen Darstellung sowie grafische Elemente zur Visualisierung (siehe beispielsweise *Anhang 10* oder *Anhang 19*)..

Tabelle 8 - FA-08

ID	FA-09
Quelle	*„Füge zusätzlich Formatierungen mit CSS ein.“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Das visuelle Design muss durch Cascading Style Sheets definiert werden.
Akzeptanzkriterium	Layout und Design (z. B. Farben, Abstände) werden nachweislich über CSS gesteuert.
Nachweis	Es wurde eine style.css (siehe *Anhang 19*) angelegt und in allen HTML-Dokumenten eingebunden (vgl. *Anhang 10*, *Anhang 11*, *Anhang 12*, *Anhang 13*, *Anhang 14*, *Anhang 15*, *Anhang 16*, *Anhang 17* und *Anhang 18*).

Tabelle 9 - FA-09

ID	FA-10
Quelle	*„Nutze JavaScript, für eine sinnvolle Suchfunktion nach relevanten Begriffen im Glossar.“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die Anwendung muss eine clientseitige Suchlogik via JavaScript besitzen.
Akzeptanzkriterium	Nutzer können Suchbegriffe eingeben und erhalten Treffer oder Links zu den Glossarseiten.
Nachweis	Diese Funktionalität wurde in der suche.js (siehe *Anhang 20*) umgesetzt und in allen Seiten eingebunden (vgl. *Anhang 10*, *Anhang 11*, *Anhang 12*, *Anhang 13*, *Anhang 14*, *Anhang 15*, *Anhang 16*, *Anhang 17* und *Anhang 18*).

Tabelle 10 - FA-10

ID	FA-11
Quelle	*„Nutze JavaScript zudem, um Datum und Uhrzeit der letzten Speicherung der jeweiligen HTML-Seite am Seitenende auszugeben.“ (IU Internationale Hochschule, n. d., S. 2)*

ID	FA-11
Beschreibung	Jede Seite muss automatisiert den Zeitstempel der letzten Änderung anzeigen.
Akzeptanzkriterium	Im Footer jeder HTML-Seite wird via JavaScript der „Last-Modified“-Zeitstempel ausgegeben.
Nachweis	Diese Funktionalität wurde in der letzteSpeicherung.js (siehe *Anhang 21*) umgesetzt und in allen Seiten eingebunden (vgl. *Anhang 10*, *Anhang 11*, *Anhang 12*, *Anhang 13*, *Anhang 14*, *Anhang 15*, *Anhang 16*, *Anhang 17* und *Anhang 18*).

Tabelle 11 - FA-11

Anhang 2 - Qualitätsanforderungen

ID	QA-01
Quelle	*„Wähle die fünf Begriffe so, dass sinnvolle Verlinkungen zwischen den Begriffen und zu externen Seiten eingefügt werden.“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die Verlinkungen sollen fachlich sinnvoll und nachvollziehbar sein.
Akzeptanzkriterium	Verlinkte Inhalte stehen in erkennbarem fachlichen Bezug zum jeweiligen Begriff und bieten einen Mehrwert für das Verständnis.
Nachweis	Die fünf Begriffe wurden aus dem Themenfeld Softwaretest so ausgewählt, dass sie fachlich aufeinander aufbauen und sinnvoll miteinander verlinkt werden können: Testbasis, Testbedingung, Testfall, Testabdeckung und Äquivalenzklassenbildung. Dadurch können Nutzer:innen direkt zwischen zusammenhängenden Begriffen navigieren, etwa von der Testbasis über Testbedingungen zum Testfall. Ergänzend wurden externe Links zu weiterführenden Fachquellen eingebunden, um zusätzliche Informationen außerhalb des Glossars bereitzustellen.

Tabelle 12 - QA-01

ID	QA-02
Quelle	*„Erstelle im Rahmen des Informationsdesigns zudem ein geeignetes Konzept für das Glossar.“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Das Informationsdesign soll für ein Glossar geeignet, verständlich und konsistent sein.
Akzeptanzkriterium	Navigation, Seitenstruktur und Inhaltsanordnung sind einheitlich gestaltet. Nutzer:innen können intuitiv zwischen den Begriffen wechseln.
Nachweis	Navigation, Seitenstruktur und Inhaltsanordnung wurden einheitlich umgesetzt, indem alle Begriffsseiten denselben Aufbau mit Logo, Suchfunktion, Rücksprung, Home-Button, Hauptüberschrift, Definition, weiterführenden Informationen und Footer verwenden. Dadurch können Nutzer:innen sich schnell orientieren und intuitiv zwischen den Begriffen sowie zurück zur Startseite wechseln.

Tabelle 13 - QA-02

ID	QA-03
Quelle	*„Nutze JavaScript, für eine sinnvolle Suchfunktion nach relevanten Begriffen im Glossar.“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die Suchfunktion soll benutzbar und zielgerichtet sein.
Akzeptanzkriterium	Relevante Begriffe werden bei Eingabe typischer Keywords (Präfix-Suche oder exakter Treffer) zuverlässig und ohne Fehlfunktionen angezeigt.
Nachweis	Die Anforderung wird erfüllt, indem JavaScript eine Live-Suche über alle Glossarbegriffe bereitstellt. Bereits während der Eingabe werden passende Treffer gefiltert und direkt als anklickbare Links angezeigt, sodass Nutzer:innen relevante Begriffe schnell, zielgerichtet und ohne Umwege finden können. Darüber hinaus wird den Nutzer:innen dargestellt, ob sie ein Begriff bereits besucht haben.

Tabelle 14 - QA-03

ID	QA-04
Beschreibung	Das Glossar soll gut wartbar und für die Belegschaft einfach erweiterbar sein.
Akzeptanzkriterium	Neue Begriffsseiten können durch Kopieren der bestehenden Struktur und Anpassung des Inhalts ohne komplexe Code-Änderungen ergänzt werden.
Nachweis	Die Anforderung wird erfüllt, indem für neue Begriffsseiten eine feste Vorlage bereitsteht, die einfach kopiert und inhaltlich angepasst werden kann. Durch die einheitliche Seitenstruktur sowie die zentrale Pflege der Begriffe in der JSON-Datei (siehe *Anhang 28*) sind dafür keine komplexen Änderungen an der übrigen Anwendung erforderlich.

Tabelle 15 - QA-04

ID	QA-05
Quelle	*„Nutze für das Glossar verschiedene Gestaltungselemente [...] Füge zusätzlich Formatierungen mit CSS ein.“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die Seiten sollen übersichtlich, lesbar und visuell strukturiert gestaltet sein.
Akzeptanzkriterium	Die Inhalte sind durch CSS-Regeln visuell gegliedert. Typografie, Abstände und Hervorhebungen unterstützen aktiv den Lesefluss.
Nachweis	CSS (siehe *Anhang 19*) erfüllt die Anforderung, indem es Inhalte klar strukturiert und visuell voneinander abgrenzt. Überschriften, Abstände, Hervorhebungen und eine konsistente Typografie sorgen dafür, dass Definitionen, Zusatzinformationen und Navigationselemente leicht erfassbar sind und den Lesefluss gezielt unterstützen.

Tabelle 16 - QA-05

ID	QA-06
Quelle	*„Dieser [Prototyp] soll dazu dienen, vor allem den Umfang der Begriffserläuterungen im Glossar abzustimmen [...]“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die Begriffserläuterungen sollen in einem angemessenen und vergleichbaren Detaillierungsgrad vorliegen.
Akzeptanzkriterium	Alle fünf Begriffsseiten weisen eine konsistente Informationstiefe auf, die als Entscheidungsgrundlage für den finalen Umfang dient.
Nachweis	Die Anforderung wird erfüllt, indem alle fünf Begriffsseiten nach demselben inhaltlichen Muster aufgebaut sind und eine vergleichbare Informationstiefe bieten. Dadurch lässt sich im Prototyp gut bewerten, welcher Umfang an Definition, Zusatzinformation und Verlinkung für die finale Glossarumsetzung sinnvoll ist.

Tabelle 17 - QA-06

Anhang 3 - Randbedingungen

ID	RB-01
Quelle	*„Erstelle einen Prototyp für das Glossar [...]“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Es ist nur ein funktionaler Prototyp gefordert, kein voll produktiver Endausbau mit Server-Infrastruktur.
Akzeptanzkriterium	Die Lösung bildet die geforderten Kernfunktionen (Begriffe, Suche, Zeitstempel) ab. Ein Live-Backend-Betrieb ist nicht erforderlich.
Nachweis	Erfüllt wird diese Anforderung durch einen statischen Prototyp, der die zentralen Funktionen des Glossars vollständig abbildet: Fünf Begriffseinträge, eine JavaScript-gestützte Suche sowie die Anzeige von Datum und Uhrzeit der letzten Speicherung. Ein Live-Backend ist dafür nicht notwendig, da die wesentlichen Anwendungsfälle bereits realistisch im Browser demonstriert werden.

Tabelle 18 - RB-01

ID	RB-02
Quelle	*„[...] für fünf selbst gewählte Begriffe, wahlweise aus den Bereichen Logistik, IT oder Marketing.“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die Anzahl der Begriffe und der fachliche Rahmen sind fest vorgegeben.
Akzeptanzkriterium	Es werden exakt fünf Begriffe aus den Clustern Logistik, IT oder Marketing verwendet.
Nachweis	Umgesetzt ist diese Anforderung durch die Auswahl von genau fünf Fachbegriffen aus dem Bereich IT, die im Glossar jeweils auf einer eigenen Unterseite erläutert werden. Damit entspricht der Prototyp dem geforderten Umfang und konzentriert sich klar auf einen thematischen Cluster.

Tabelle 19 - RB-02

ID	RB-03
Quelle	*„Erstelle das Glossar in HTML. [...] Füge zusätzlich Formatierungen mit CSS ein. Nutze JavaScript [...]“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die technische Umsetzung ist auf die Web-Standards HTML, CSS und JavaScript festgelegt.
Akzeptanzkriterium	Die Webanwendung nutzt HTML für die Struktur, CSS für das Design und JavaScript für die geforderte Logik.
Nachweis	Für die Umsetzung wurden HTML, CSS und Javascript eingesetzt (siehe *Anhang 10* bis *Anhang 27*).

Tabelle 20 - RB-03

ID	RB-04
Quelle	*„Nutze für die Beschreibung der erstellten Website in der Fallstudie geeignete Screenshots.“ (IU Internationale Hochschule, n. d., S. 4)*
Beschreibung	Die schriftliche Ausarbeitung muss die visuelle Umsetzung belegen.
Akzeptanzkriterium	Die Fallstudie enthält aussagekräftige Screenshots aller wesentlichen Unterseiten und Funktionen (z. B. Suchergebnisse).

ID	RB-04
Nachweis	Die Fallstudie enthält mehrere aussagekräftige Screenshots (siehe *Anhang 4 bis Anhang 9*).

Tabelle 21 - RB-04

ID	RB-05
Quelle	*„Erstelle den Quellcode selbst in einem Editor.“ (IU Internationale Hochschule, n. d., S. 4)*
Beschreibung	Der Quellcode muss durch eigenhändige Programmierung in einem Editor entstehen.
Akzeptanzkriterium	Der Code weist eine individuelle, händische Struktur auf und ist editorbasiert erstellt.
Nachweis	Der Code wurde manuell in Microsoft Visual Studio Code entwickelt, die Struktur wurde selbstständig gewählt und erstellt.

Tabelle 22 - RB-05

ID	RB-06
Quelle	*„Nutze keine Tools wie Frontpage [...] und auch keine Content Management Systeme wie WordPress.“ (IU Internationale Hochschule, n. d., S. 4)*
Beschreibung	WYSIWYG-Editoren und CMS-Generatoren sind für die Erstellung explizit ausgeschlossen.
Akzeptanzkriterium	Es kommen weder Frontpage noch vergleichbare grafische Editoren oder CMS wie WordPress zum Einsatz.
Nachweis	Für die Umsetzung wurde Microsoft Visual Studio Code verwendet.

Tabelle 23 - RB-06

ID	RB-07
Quelle	*„Der komplette Quellcode muss in den Anhang der Arbeit einfügt werden.“ (IU Internationale Hochschule, n. d., S. 4)*
Beschreibung	Der vollständige Quellcode aller Dateien ist integraler Bestandteil der Abgabe.
Akzeptanzkriterium	Der Anhang der Fallstudie enthält den vollständigen HTML-, CSS- und JavaScript-Code.
Nachweis	Der Code befindet sich im *Anhang 10* bis *Anhang 28* und auf GitHub.com *https://github.com/chrisoel/DLBECGP01*.

Tabelle 24 - RB-07

ID	RB-08
Quelle	*„[...] kann der Quellcode im Anhang auch in einer kleineren Schriftgröße dargestellt werden – beispielsweise Arial 8Pkt.“ (IU Internationale Hochschule, n. d., S. 4)*
Beschreibung	Für den umfangreichen Codeanhang ist eine reduzierte Schriftgröße zur Platzersparnis zulässig.
Akzeptanzkriterium	Der Codeanhang ist lesbar eingebunden und die ggf. verringerte Schriftgröße wird ausschließlich dort angewendet.
Nachweis	Der Code befindet sich im *Anhang 10* bis *Anhang 28* und auf GitHub.com *https://github.com/chrisoel/DLBECGP01*.

Tabelle 25 - RB-08

ID	RB-09
Quelle	*„Bei der Analyse der Fallstudie und ihrer Bearbeitung müssen die im Prüfungsleitfaden aufgeführten Bewertungskriterien [...] berücksichtigt werden.“ (IU Internationale Hochschule, n. d., S. 5)*
Beschreibung	Die Bearbeitung muss sich zwingend an den offiziellen Bewertungskriterien der IU orientieren.
Akzeptanzkriterium	Die Ausarbeitung folgt strukturell und inhaltlich den im Prüfungsleitfaden geforderten Qualitätsstandards.
Nachweis	Es erfolgte die Ausarbeitung gemäß des Prüfungsleitfadens für Fallstudien vor dem 01.04.2026.

Tabelle 26 - RB-09

ID	RB-10
Quelle	*„Die Mitarbeiter:innen der Log7-Consult GmbH sollen zukünftig eigene Erweiterungen am Glossar vornehmen können. Erläutere hierfür relevante Befehle von HTML, CSS und JavaScript, die Du verwendet hast. “ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Die bei der Umsetzung eingesetzten Grundlagen aus HTML, CSS und JavaScript sind dokumentiert und erläutert.
Akzeptanzkriterium	Der Code und die Fallstudie enthalten Erläuterungen zu den zentralen verwendeten Befehlen und Funktionen.
Nachweis	Im Code wurden prägnante Kommentare hinzugefügt und in der Wartungsanleitung (siehe *Anhang 29*) der Prozess zum Hinzufügen eines neuen Begriffs erläutert, um die Wartbarkeit zu erhöhen. Die vollständige Wartungsanleitung befindet sich als PDF und Markdown Datei auf Github.com.

Tabelle 27 - RB-10

ID	RB-11
Quelle	*„Triff Annahmen zu den zukünftigen Anforderungen [...] und wie ein Prozess zur Aktualisierung [...] gestaltet werden kann.“ (IU Internationale Hochschule, n. d., S. 2)*
Beschreibung	Es müssen Empfehlungen für den weiteren Betrieb und Pflegeprozesse erstellt werden.
Akzeptanzkriterium	Die Dokumentation enthält einen strategischen Ausblick und eine Prozessbeschreibung für Inhalts-Updates.
Nachweis	Ein Ausblick befindet sich in *Kapitel 7 „Ergebnis & Fazit“*.

Tabelle 28 - RB-11

Anhang 4 - Abbildung index.html

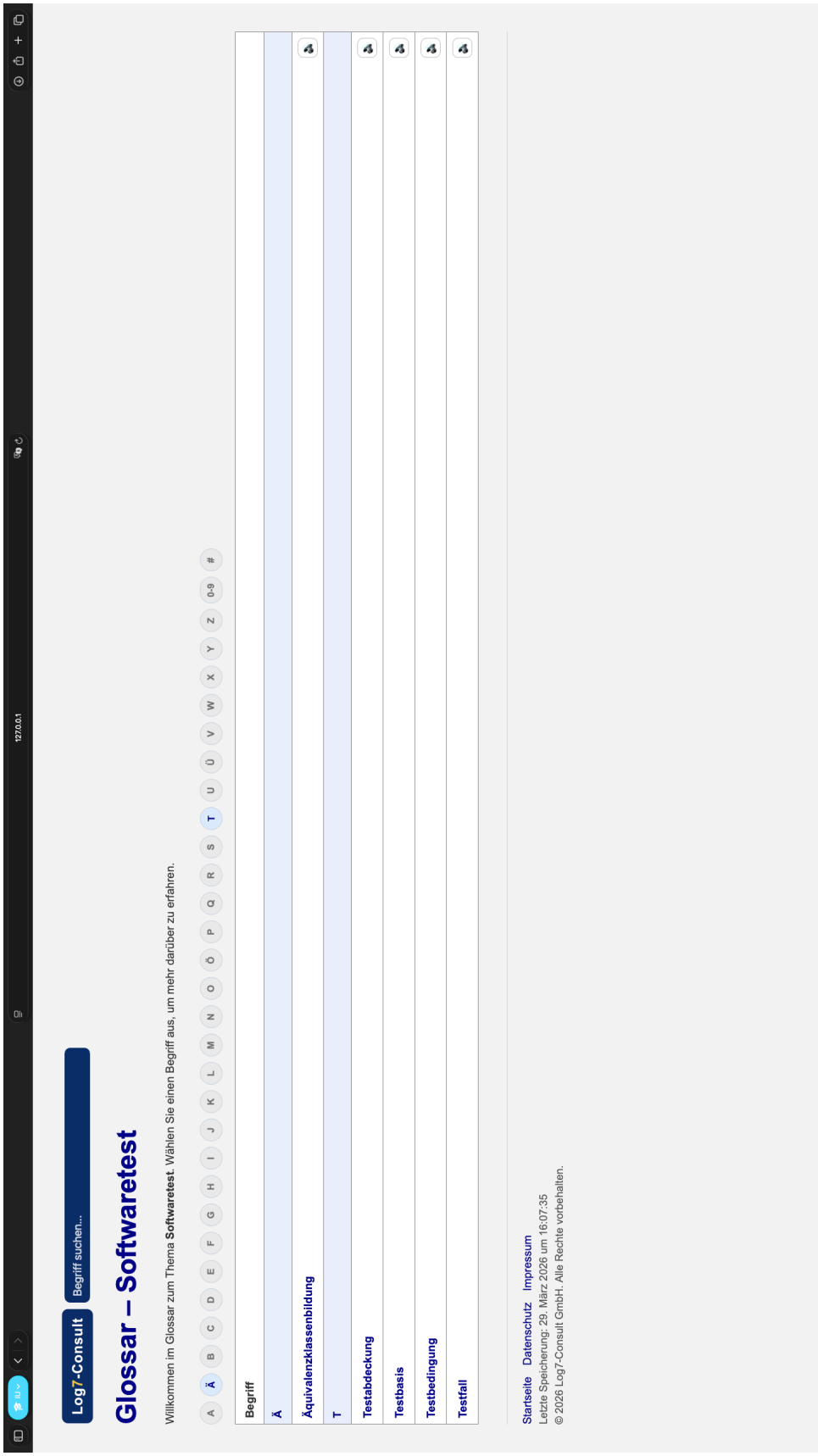

Abbildung 1 - index.html

Anhang 5 - Abbildung Such-Funktion

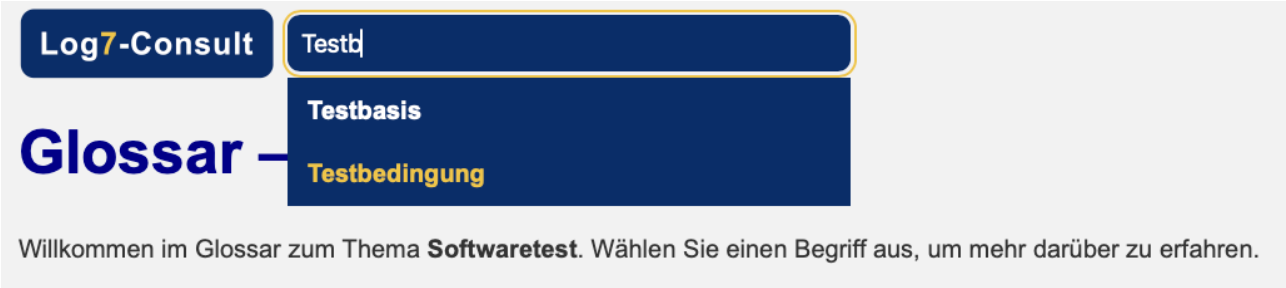

Abbildung 2 - Such-Funktion

Anhang 6 - Abbildung Zeitstempel-Funktion

Abbildung 3 - Zeitstempel-Funktion

Anhang 7 - Abbildung Begriffsseite

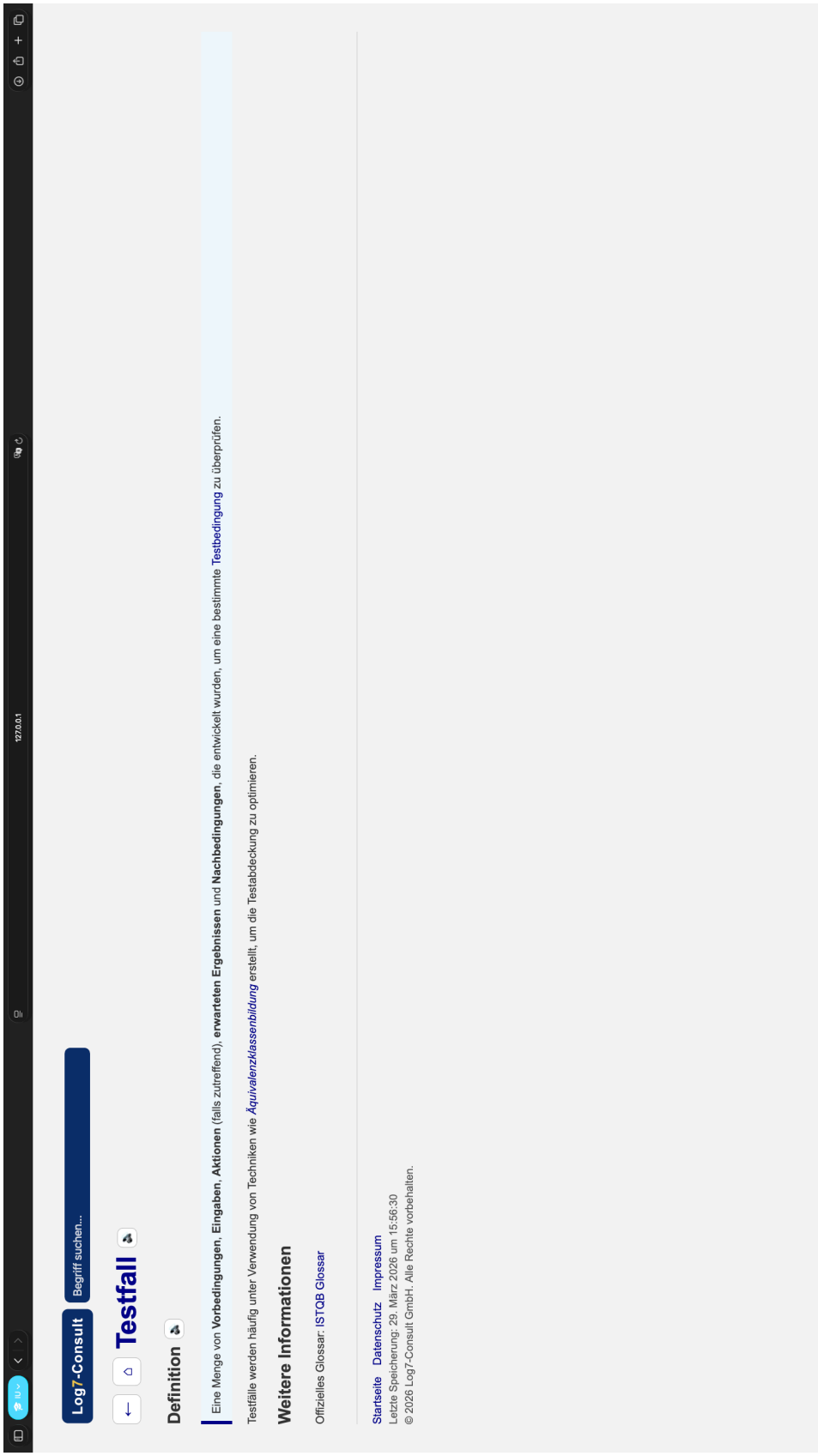

Abbildung 4 - Begriffsseite

Anhang 8 - Abbildung Begriffseite-Navigation

Abbildung 5 - Begriffseite-Navigation

Anhang 9 - Abbildung Verlinkung

Definition

> Ein prüfbares Element eines Systems oder einer Komponente, das durch einen oder mehrere **Testfälle** verifiziert werden kann. Testbedingungen werden aus der *Testbasis* abgeleitet.

Weitere Informationen

Offizielles Glossar: ISTQB Glossar

Abbildung 6 - Verlinkung

Anhang 10 - index.html

```
<!DOCTYPE html>
<!--
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
-->
<html lang="de">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Glossar – Softwaretest</title>
    <link rel="stylesheet" href="style.css">
    <link rel="icon" href="favicon.svg" type="image/svg+xml">
</head>

<body>

    <!-- Logo und Suche werden per JavaScript eingesetzt -->
    <div id="site-topbar" data-base=""></div>

    <!-- Kopfbereich der Seite -->
    <div class="page-header">
        <h1>Glossar – Softwaretest</h1>
    </div>

    <p class="intro">
        Willkommen im Glossar zum Thema <strong>Softwaretest</strong>.
        Wählen Sie einen Begriff aus, um mehr darüber zu erfahren.
    </p>

    <!-- Filterleiste und Tabelle werden aus der JSON-Datei aufgebaut -->
    <nav class="alphabet-nav" id="alphabet-nav" aria-label="Sprungnavigation zu den Begriffsgruppen"></nav>

    <table class="glossary-table">
        <thead>
            <tr>
                <th scope="col">Begriff</th>
            </tr>
        </thead>
        <tbody id="glossary-table-body"></tbody>
    </table>

    <!-- Der gemeinsame Footer wird per JavaScript eingesetzt -->
    <footer id="site-footer" data-base=""></footer>

    <!-- Reihenfolge: JSON-Daten, Hilfsfunktionen, Branding, Seitenskripte -->
    <script src="js/begriffeDaten.js"></script>
    <script src="js/logo.js"></script>
    <script src="js/vorlesen.js"></script>
    <script src="js/suche.js"></script>
    <script src="js/startseiteGlossar.js"></script>
```

```
    <script src="js/footer.js"></script>
    <script src="js/letzteSpeicherung.js"></script>

</body>
</html>
```

Anhang 11 - testfall.html

```
<!DOCTYPE html>
<!--
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
-->
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Testfall – Glossar</title>
  <link rel="stylesheet" href="../style.css">
  <link rel="icon" href="../favicon.svg" type="image/svg+xml">
</head>

<body>

  <!-- Logo und Suche werden per JavaScript eingesetzt -->
  <div id="site-topbar" data-base="../"></div>

  <!-- Seitentitel mit Navigation -->
  <div class="page-header">
    <div class="title-group">
      <a class="back-link" href="../index.html" data-back-link aria-label="Zur vorherigen Seite">&#8592;</a>
      <a class="home-link" href="../index.html" aria-label="Zur Startseite">&#8962;</a>
      <h1>Testfall</h1>
    </div>
  </div>

  <!-- Fachinhalt des Begriffs -->
  <h2>Definition</h2>
  <p class="highlight">
    Eine Menge von <strong>Vorbedingungen</strong>, <strong>Eingaben</strong>,
    <strong>Aktionen</strong> (falls zutreffend), <strong>erwarteten Ergebnissen</strong>
    und <strong>Nachbedingungen</strong>, die entwickelt wurden, um eine bestimmte
    <a href="testbedingung.html">Testbedingung</a> zu überprüfen.
  </p>

  <p>
    Testfälle werden häufig unter Verwendung von Techniken wie
    <em><a href="aequivalenzklassenbildung.html">Äquivalenzklassenbildung</a></em> erstellt,
    um die Testabdeckung zu optimieren.
  </p>

  <h2>Weitere Informationen</h2>
  <p>
    Offizielles Glossar:
    <a href="https://glossary.istqb.org/de_DE/term/testfall?term=Testfall&exact_matches_first=true">
      ISTQB Glossar
    </a>
  </p>
```

```
    <!-- Der gemeinsame Footer wird per JavaScript eingesetzt -->
    <footer id="site-footer" data-base="../"></footer>

    <!-- Reihenfolge: JSON-Daten, Branding, Vorlesen, Suche, Navigation, gemeinsame Seitenelemente -->
    <script src="../js/begriffeDaten.js"></script>
    <script src="../js/logo.js"></script>
    <script src="../js/vorlesen.js"></script>
    <script src="../js/suche.js"></script>
    <script src="../js/zurueckLink.js"></script>
    <script src="../js/footer.js"></script>
    <script src="../js/letzteSpeicherung.js"></script>
</body>
</html>
```

Anhang 12 - testbedingung.html

```
<!DOCTYPE html>
<!--
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
-->
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Testbedingung – Glossar</title>
  <link rel="stylesheet" href="../style.css">
  <link rel="icon" href="../favicon.svg" type="image/svg+xml">
</head>

<body>

  <!-- Logo und Suche werden per JavaScript eingesetzt -->
  <div id="site-topbar" data-base="../"></div>

  <!-- Seitentitel mit Navigation -->
  <div class="page-header">
    <div class="title-group">
      <a class="back-link" href="../index.html" data-back-link aria-label="Zur vorherigen Seite">&#8592;</a>
      <a class="home-link" href="../index.html" aria-label="Zur Startseite">&#8962;</a>
      <h1>Testbedingung</h1>
    </div>
  </div>

  <!-- Fachinhalt des Begriffs -->
  <h2>Definition</h2>
  <p class="highlight">
    Ein prüfbares Element eines Systems oder einer Komponente, das durch einen oder mehrere
    <strong><a href="testfall.html">Testfälle</a></strong> verifiziert werden kann.
    Testbedingungen werden aus der <em><a href="testbasis.html">Testbasis</a></em> abgeleitet.
  </p>

  <h2>Weitere Informationen</h2>
  <p>
    Offizielles Glossar:
    <a href="https://glossary.istqb.org/de_DE/term/testbedingung?term=Testbedingung&exact_matches_first=true">
      ISTQB Glossar
    </a>
  </p>

  <!-- Der gemeinsame Footer wird per JavaScript eingesetzt -->
  <footer id="site-footer" data-base="../"></footer>

  <!-- Reihenfolge: JSON-Daten, Branding, Vorlesen, Suche, Navigation, gemeinsame Seitenelemente -->
  <script src="../js/begriffeDaten.js"></script>
  <script src="../js/logo.js"></script>
```

```
    <script src="../js/vorlesen.js"></script>
    <script src="../js/suche.js"></script>
    <script src="../js/zurueckLink.js"></script>
    <script src="../js/footer.js"></script>
    <script src="../js/letzteSpeicherung.js"></script>
</body>
</html>
```

Anhang 13 - testbasis.html

```
<!DOCTYPE html>
<!--
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
-->
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Testbasis – Glossar</title>
  <link rel="stylesheet" href="../style.css">
  <link rel="icon" href="../favicon.svg" type="image/svg+xml">
</head>

<body>

  <!-- Logo und Suche werden per JavaScript eingesetzt -->
  <div id="site-topbar" data-base="../"></div>

  <!-- Seitentitel mit Navigation -->
  <div class="page-header">
    <div class="title-group">
      <a class="back-link" href="../index.html" data-back-link aria-label="Zur vorherigen Seite">&#8592;</a>
      <a class="home-link" href="../index.html" aria-label="Zur Startseite">&#8962;</a>
      <h1>Testbasis</h1>
    </div>
  </div>

  <!-- Fachinhalt des Begriffs -->
  <h2>Definition</h2>
  <p class="highlight">
    Alle <strong>Artefakte</strong>, aus denen die Anforderungen an eine Komponente oder ein System
    abgeleitet werden können. Aus der Testbasis werden
    <strong><a href="testbedingung.html">Testbedingungen</a></strong> abgeleitet,
    die durch <em>Testfälle</em> überprüft werden.
  </p>

  <h2>Weitere Informationen</h2>
  <p>
    Siehe auch das
    <a href="https://glossary.istqb.org/de_DE/term/testbasis?term=Testbasis&exact_matches_first=true">
      ISTQB Glossar
    </a>.
  </p>

  <!-- Der gemeinsame Footer wird per JavaScript eingesetzt -->
  <footer id="site-footer" data-base="../"></footer>

  <!-- Reihenfolge: JSON-Daten, Branding, Vorlesen, Suche, Navigation, gemeinsame Seitenelemente -->
  <script src="../js/begriffeDaten.js"></script>
```

```
    <script src="../js/logo.js"></script>
    <script src="../js/vorlesen.js"></script>
    <script src="../js/suche.js"></script>
    <script src="../js/zurueckLink.js"></script>
    <script src="../js/footer.js"></script>
    <script src="../js/letzteSpeicherung.js"></script>
</body>
</html>
```

Anhang 14 - testabdeckung.html

```
<!DOCTYPE html>
<!--
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
-->
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Testabdeckung – Glossar</title>
  <link rel="stylesheet" href="../style.css">
  <link rel="icon" href="../favicon.svg" type="image/svg+xml">
</head>

<body>

  <!-- Logo und Suche werden per JavaScript eingesetzt -->
  <div id="site-topbar" data-base="../"></div>

  <!-- Seitentitel mit Navigation -->
  <div class="page-header">
    <div class="title-group">
      <a class="back-link" href="../index.html" data-back-link aria-label="Zur vorherigen Seite">&#8592;</a>
      <a class="home-link" href="../index.html" aria-label="Zur Startseite">&#8962;</a>
      <h1>Testabdeckung</h1>
    </div>
  </div>

  <!-- Fachinhalt des Begriffs -->
  <h2>Definition</h2>
  <p class="highlight">
    Der Grad, zu dem <strong><a href="testbasis.html">Testbasis</a></strong>-Elemente
    durch eine Testmenge überprüft werden, ausgedrückt als <strong>Prozentsatz</strong>.
    Eine hohe Testabdeckung zeigt, dass die definierten
    <strong><a href="testfall.html">Testfälle</a></strong> die identifizierten
    <em><a href="testbedingung.html">Testbedingungen</a></em> gut abdecken.
  </p>

  <div class="note">
    <strong>Hinweis:</strong> Im offiziellen ISTQB-Glossar wird dieser Begriff als
    „Überdeckung“ bezeichnet. Für dieses Glossar verwenden wir die Bezeichnung
    „Testabdeckung“, da sie im deutschen Kontext üblicher ist.
  </div>

  <h2>Weitere Informationen</h2>
  <p>
    Offizielles Glossar:
    <a href="https://glossary.istqb.org/de_DE/term/uberdeckung?term=Abdeckung&exact_matches_first=true">
      ISTQB Glossar
    </a>
```

```
    </p>

    <!-- Der gemeinsame Footer wird per JavaScript eingesetzt -->
    <footer id="site-footer" data-base="../"></footer>

    <!-- Reihenfolge: JSON-Daten, Branding, Vorlesen, Suche, Navigation, gemeinsame Seitenelemente -->
    <script src="../js/begriffeDaten.js"></script>
    <script src="../js/logo.js"></script>
    <script src="../js/vorlesen.js"></script>
    <script src="../js/suche.js"></script>
    <script src="../js/zurueckLink.js"></script>
    <script src="../js/footer.js"></script>
    <script src="../js/letzteSpeicherung.js"></script>
</body>
</html>
```

Anhang 15 - aequivalenzklassenbildung.html

```
<!DOCTYPE html>
<!--
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
-->
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Äquivalenzklassenbildung – Glossar</title>
  <link rel="stylesheet" href="../style.css">
  <link rel="icon" href="../favicon.svg" type="image/svg+xml">
</head>

<body>

  <!-- Logo und Suche werden per JavaScript eingesetzt -->
  <div id="site-topbar" data-base="../"></div>

  <!-- Seitentitel mit Navigation -->
  <div class="page-header">
    <div class="title-group">
      <a class="back-link" href="../index.html" data-back-link aria-label="Zur vorherigen Seite">&#8592;</a>
      <a class="home-link" href="../index.html" aria-label="Zur Startseite">&#8962;</a>
      <h1>Äquivalenzklassenbildung</h1>
    </div>
  </div>

  <!-- Fachinhalt des Begriffs -->
  <h2>Definition</h2>
  <p class="highlight">
    Ein <strong>Black-Box-Testentwurfsverfahren</strong>, bei dem Eingabedaten in
    <strong>Partitionen (Äquivalenzklassen)</strong> unterteilt werden, von denen angenommen wird,
    dass sie sich gleich verhalten. Dadurch können
    <strong><a href="testfall.html">Testfälle</a></strong> aus jeder Partition abgeleitet werden.
    Dieses Verfahren erhöht die Effizienz der Testfallgestaltung und verbessert die
    <em><a href="testabdeckung.html">Testabdeckung</a></em>.
  </p>

  <h2>Weitere Informationen</h2>
  <p>
    Offizielles Glossar:
    <a href="https://glossary.istqb.org/de_DE/term/aquivalenzklassenbildung?term=Äquivalenzklassenbildung&exact_matches_first=true">
      ISTQB Glossar
    </a>
  </p>

  <!-- Der gemeinsame Footer wird per JavaScript eingesetzt -->
  <footer id="site-footer" data-base="../"></footer>
```

```
    <!-- Reihenfolge: JSON-Daten, Branding, Vorlesen, Suche, Navigation, gemeinsame Seitenelemente -->
    <script src="../js/begriffeDaten.js"></script>
    <script src="../js/logo.js"></script>
    <script src="../js/vorlesen.js"></script>
    <script src="../js/suche.js"></script>
    <script src="../js/zurueckLink.js"></script>
    <script src="../js/footer.js"></script>
    <script src="../js/letzteSpeicherung.js"></script>
</body>
</html>
```

Anhang 16 - _vorlage.html

```
<!DOCTYPE html>
<!--
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
-->
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <!-- Anpassen: Begriff im Seitentitel einsetzen -->
  <title>Neuer Begriff – Glossar</title>
  <link rel="stylesheet" href="../style.css">
  <link rel="icon" href="../favicon.svg" type="image/svg+xml">
</head>

<body>

  <div id="site-topbar" data-base="../"></div>

  <!-- Anpassbarer Bereich: Seitentitel, Hauptüberschrift und Seiteninhalt -->
  <div class="page-header">
    <div class="title-group">
      <a class="back-link" href="../index.html" data-back-link aria-label="Zur vorherigen Seite">&#8592;</a>
      <a class="home-link" href="../index.html" aria-label="Zur Startseite">&#8962;</a>
      <h1>Neuer Begriff</h1>
    </div>
  </div>

  <h2>Definition</h2>
  <p class="highlight">
    Hier steht die Definition des neuen Begriffs. Ein Verweis auf einen anderen
    Glossarbegriff kann direkt im Text gesetzt werden, zum Beispiel
    <a href="testfall.html">Testfall</a>.
  </p>

  <!-- Zusätzliche Inhalte wie Hinweise oder Quellen können darunter ergänzt werden -->
  <h2>Weitere Informationen</h2>
  <p>
    Quelle oder weiterführende Informationen hier ergänzen.
  </p>
  <!-- Ende anpassbarer Bereich -->

  <footer id="site-footer" data-base="../"></footer>

  <script src="../js/begriffeDaten.js"></script>
  <script src="../js/logo.js"></script>
  <script src="../js/vorlesen.js"></script>
  <script src="../js/suche.js"></script>
  <script src="../js/zurueckLink.js"></script>
  <script src="../js/footer.js"></script>
```

```
    <script src="../js/letzteSpeicherung.js"></script>
</body>
</html>
```

Anhang 17 - datenschutz.html

```
<!DOCTYPE html>
<!--
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
-->
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Datenschutz – Glossar</title>
  <link rel="stylesheet" href="style.css">
  <link rel="icon" href="favicon.svg" type="image/svg+xml">
</head>

<body>

  <!-- Logo und Suche werden per JavaScript eingesetzt -->
  <div id="site-topbar" data-base=""></div>

  <!-- Kopfbereich der Seite -->
  <div class="page-header">
    <h1>Datenschutz</h1>
  </div>

  <!-- Platzhaltertext für die Fallstudie -->
  <main>
    <p class="note">
      Diese Datenschutzseite ist im Rahmen der Fallstudie <strong>out of scope</strong> und dient daher nur als Platzhalter.
    </p>

    <p>
      Im realen Einsatz muss hier eine vollständige und rechtlich korrekte Datenschutzerklärung hinterlegt werden,
      die die tatsächliche Verarbeitung personenbezogener Daten transparent beschreibt.
    </p>
  </main>

  <!-- Der gemeinsame Footer wird per JavaScript eingesetzt -->
  <footer id="site-footer" data-base=""></footer>

  <!-- Reihenfolge: JSON-Daten, Branding, Suche, gemeinsame Seitenelemente -->
  <script src="js/begriffeDaten.js"></script>
  <script src="js/logo.js"></script>
  <script src="js/suche.js"></script>
  <script src="js/footer.js"></script>
  <script src="js/letzteSpeicherung.js"></script>
</body>
</html>
```

Anhang 18 - impressum.html

```
<!DOCTYPE html>
<!--
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
-->
<html lang="de">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Impressum – Glossar</title>
   <link rel="stylesheet" href="style.css">
   <link rel="icon" href="favicon.svg" type="image/svg+xml">
</head>

<body>

   <!-- Logo und Suche werden per JavaScript eingesetzt -->
   <div id="site-topbar" data-base=""></div>

   <!-- Kopfbereich der Seite -->
   <div class="page-header">
      <h1>Impressum</h1>
   </div>

   <!-- Platzhaltertext für die Fallstudie -->
   <main>
      <p class="note">
         Dieses Impressum ist im Rahmen der Fallstudie <strong>out of scope</strong> und wird daher bewusst nur als Platzhalter geführt.
      </p>

      <p>
         Im realen Einsatz muss an dieser Stelle ein vollständiges und rechtlich korrektes Impressum mit allen
         erforderlichen Angaben veröffentlicht werden.
      </p>
   </main>

   <!-- Der gemeinsame Footer wird per JavaScript eingesetzt -->
   <footer id="site-footer" data-base=""></footer>

   <!-- Reihenfolge: JSON-Daten, Branding, Suche, gemeinsame Seitenelemente -->
   <script src="js/begriffeDaten.js"></script>
   <script src="js/logo.js"></script>
   <script src="js/suche.js"></script>
   <script src="js/footer.js"></script>
   <script src="js/letzteSpeicherung.js"></script>
</body>
</html>
```

Anhang 19 - style.css

```
/*
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
*/

/* Design-Tokens */
:root {
    --color-page-bg: #f5f5f5;
    --color-surface: white;
    --color-surface-muted: #f0f0f0;
    --color-surface-soft: #ececec;
    --color-brand-soft: #dce9ff;
    --color-brand-soft-hover: #c7dcff;
    --color-brand: darkblue;
    --color-brand-strong: #0b2f6b;
    --color-brand-accent: #f0c84b;
    --color-heading: #333;
    --color-text-muted: #555;
    --color-text-disabled: #777;
    --color-border-default: #bbb;
    --color-border-light: #ccc;
    --color-border-soft: #c9d3df;
    --color-border-note: gray;
    --color-highlight-bg: #f0f8ff;
    --color-note-bg: #f9f9f9;
    --color-error: #8a1f11;
    --color-filter-active: #ffe36b;
    --color-filter-active-border: #d3a700;
    --shadow-filter-active: 0 0 10px rgba(255, 215, 0, 0.75);
    --radius-large: 10px;
    --radius-medium: 8px;
    --space-page: 40px;
    --space-page-mobile: 20px;
    --space-section: 20px;
    --space-content: 10px;
}

/* Basislayout */
body {
    font-family: Arial, sans-serif;
    margin: var(--space-page);
    background-color: var(--color-page-bg);
    color: var(--color-heading);
    line-height: 1.5;
}

a {
    color: var(--color-brand);
    text-decoration: none;
}
```

```
a:hover {
    text-decoration: underline;
}

h1 {
    margin: 0;
    text-align: left;
    color: var(--color-brand);
    font-size: clamp(1.8rem, 3.2vw, 2.4rem);
}

h2 {
    margin-top: 0;
    color: var(--color-heading);
    font-size: clamp(1.2rem, 2.2vw, 1.5rem);
}

p,
li,
td,
th {
    overflow-wrap: break-word;
}

/* Branding */
.site-topbar {
    display: flex;
    align-items: center;
    justify-content: flex-start;
    flex-wrap: wrap;
    gap: 8px;
    margin-bottom: 18px;
}

.site-topbar .search-box {
    flex: 0 1 360px;
    max-width: 360px;
}

.site-logo {
    display: inline-flex;
    align-items: center;
    text-decoration: none;
}

.site-logo-image {
    display: block;
    height: 46px;
    width: auto;
}

.site-logo:hover {
    text-decoration: none;
```

```
}

/* Kopfbereich */
.page-header {
    display: flex;
    align-items: center;
    flex-wrap: wrap;
    gap: 12px;
    margin-bottom: var(--space-section);
}

.title-group,
.glossary-entry,
.definition-heading {
    display: flex;
    align-items: center;
}

.title-group {
    flex-wrap: wrap;
    gap: 12px;
}

.title-group h1 {
    flex: 1 1 auto;
    min-width: 0;
}

.search-box {
    position: relative;
    width: min(360px, 100%);
}

.home-link,
.back-link,
.read-aloud-button {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    box-sizing: border-box;
    border: 1px solid var(--color-border-default);
    border-radius: var(--radius-medium);
    background-color: var(--color-surface);
    color: var(--color-brand);
    line-height: 1;
}

.home-link,
.back-link {
    min-width: 40px;
    height: 40px;
    padding: 0 10px;
    font-size: 1.3rem;
}
```

```
.back-link:hover,
.home-link:hover,
.read-aloud-button:hover:not(:disabled) {
    background-color: var(--color-surface-muted);
    text-decoration: none;
}

/* Suchbereich */
#search {
    width: 100%;
    padding: 8px;
    box-sizing: border-box;
    border: 1px solid var(--color-brand-strong);
    border-radius: var(--radius-medium);
    background-color: var(--color-brand-strong);
    color: var(--color-surface);
    caret-color: var(--color-surface);
    font-size: 16px;
}

#search::placeholder {
    color: rgba(255, 255, 255, 0.8);
}

#search:focus {
    outline: 2px solid var(--color-brand-accent);
    outline-offset: 2px;
}

#results {
    position: absolute;
    top: 100%;
    left: 0;
    z-index: 10;
    width: 100%;
    margin: 4px 0 0;
    padding: 0;
    list-style: none;
    box-sizing: border-box;
    border: 1px solid var(--color-brand-strong);
    background-color: var(--color-brand-strong);

    /* Höhe für genau drei sichtbare Treffer, weitere bleiben scrollbar. */
    max-height: calc(3 * 40px + 2px);
    overflow-y: auto;
}

#results:empty {
    display: none;
}

#results li {
    margin: 0;
```

```
}

#results li a {
    display: block;
    padding: 10px 12px;
    line-height: 20px;
    color: var(--color-brand-accent);
    font-weight: 600;

    /* Lange Begriffe sollen die Liste nicht verbreitern. */
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
}

#results li a:visited {
    color: var(--color-surface);
}

#results li:hover {
    background-color: rgba(255, 255, 255, 0.08);
}

.search-error {
    margin: 6px 0 0;
    color: var(--color-error);
    font-size: 0.9rem;
    font-weight: 600;
}

/* Startseite */
.intro {
    margin-bottom: 30px;
    text-align: left;
}

.alphabet-nav {
    display: flex;
    flex-wrap: wrap;
    gap: 8px;
    margin-bottom: 18px;
}

.alphabet-link,
.alphabet-label {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    min-width: 32px;
    min-height: 32px;
    padding: 0 8px;
    box-sizing: border-box;
    border: 1px solid var(--color-border-soft);
    border-radius: 999px;
```

```
    font-size: 0.85rem;
    font-weight: bold;
}

.alphabet-link {
    background-color: var(--color-brand-soft);
    color: var(--color-brand);
    cursor: pointer;
}

.alphabet-link:hover {
    background-color: var(--color-brand-soft-hover);
    text-decoration: none;
}

.alphabet-link.is-active {
    border: 3px solid var(--color-filter-active-border);
    background-color: var(--color-filter-active);
    box-shadow: var(--shadow-filter-active);
}

.alphabet-label {
    background-color: var(--color-surface-soft);
    color: var(--color-text-disabled);
}

.glossary-table {
    margin-top: 0;
}

.glossary-table th[scope="row"] {
    width: auto;
}

.glossary-entry,
.definition-heading {
    gap: 10px;
}

.glossary-entry {
    justify-content: space-between;
}

.glossary-entry a {
    flex: 1 1 auto;
    min-width: 0;
}

.read-aloud-button {
    min-width: 28px;
    height: 28px;
    padding: 0;
    cursor: pointer;
    font-size: 0.8rem;
```

```
}

.title-read-button {
    flex-shrink: 0;
}

.read-aloud-button:disabled {
    cursor: not-allowed;
    opacity: 0.55;
}

.glossary-letter-row th {
    background-color: #e9f1ff;
    color: var(--color-brand);
    font-size: 0.95rem;
}

/* Inhaltselemente */
.highlight {
    padding: var(--space-content);
    border-left: 5px solid var(--color-brand);
    background-color: var(--color-highlight-bg);
}

.note {
    margin-top: 10px;
    padding: var(--space-content);
    border-left: 5px solid var(--color-border-note);
    background-color: var(--color-note-bg);
}

table {
    width: 100%;
    margin-top: 15px;
    border-collapse: collapse;
    background-color: var(--color-surface);
}

table,
th,
td {
    border: 1px solid var(--color-border-note);
}

th,
td {
    padding: 8px;
    text-align: left;
}

/* Footer */
.page-footer {
    margin-top: 40px;
    padding-top: 16px;
```

```
    border-top: 1px solid var(--color-border-light);
}

.page-footer p {
    margin: 0;
}

.footer-links {
    display: flex;
    flex-wrap: wrap;
    gap: 16px;
}

.last-modified,
.footer-copyright {
    color: var(--color-text-muted);
    font-size: 0.95rem;
}

.last-modified {
    margin-top: 8px;
}

.footer-copyright {
    margin-top: 6px;
}

/* Responsive Anpassungen */
@media (max-width: 900px) {
    body {
        margin: 28px;
    }

    .site-topbar {
        align-items: stretch;
    }

    .site-topbar .search-box {
        flex: 1 1 100%;
        max-width: none;
    }

    .title-group {
        width: 100%;
    }

    .title-group h1 {
        width: 100%;
    }

    .glossary-entry {
        gap: 8px;
    }
}
```

```
@media (max-width: 600px) {
  body {
    margin: var(--space-page-mobile);
  }

  .site-topbar {
    gap: 10px;
  }

  .site-logo {
    width: 100%;
    justify-content: center;
  }

  .site-logo-image {
    height: 42px;
  }

  .site-topbar .search-box,
  .title-group,
  .search-box {
    width: 100%;
  }

  .title-group {
    gap: 8px;
    align-items: flex-start;
  }

  .title-group h1 {
    width: calc(100% - 88px);
    font-size: clamp(1.5rem, 7vw, 1.9rem);
  }

  .definition-heading,
  .glossary-entry,
  .footer-links {
    gap: 8px;
  }

  .home-link,
  .back-link {
    min-width: 36px;
    height: 36px;
    padding: 0 8px;
    font-size: 1.1rem;
  }

  .alphabet-link,
  .alphabet-label {
    min-width: 30px;
    min-height: 30px;
    font-size: 0.8rem;
```

```
    }

    #search {
        padding: 10px;
    }

    #results li a {
        padding: 12px;
    }

    .highlight,
    .note {
        padding: 12px;
    }

    th,
    td {
        padding: 6px;
    }
}
```

Anhang 20 - suche.js

```
/*
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
*/

/*
Zweck:
- Verknüpft das Suchfeld in der Topbar mit den Glossarbegriffen aus der JSON.
- Erstellt die Trefferliste live während der Eingabe.

Abhängigkeit:
- `window.glossarDatenPromise` aus `begriffeDaten.js`
*/

const appBaseUrl = new URL("../", document.currentScript.src);

// Wandelt relative Pfade so um, dass Suche im Root und im Begriffsordner gleich funktioniert.
function resolveAppPath(path) {
    return new URL(path, appBaseUrl).toString();
}

document.addEventListener("DOMContentLoaded", async () => {
    const searchInput = document.getElementById("search");
    const resultsList = document.getElementById("results");

    if (!searchInput || !resultsList || !window.glossarDatenPromise) {
        return;
    }

    // Blendet einen sichtbaren Fehlerhinweis direkt unter dem Suchfeld ein.
    function zeigeSucheFehler(text) {
        const searchBox = searchInput.closest(".search-box");

        if (!searchBox) {
            return;
        }

        let fehlerHinweis = searchBox.querySelector(".search-error");

        if (!fehlerHinweis) {
            fehlerHinweis = document.createElement("p");
            fehlerHinweis.className = "search-error";
            fehlerHinweis.setAttribute("role", "alert");
            searchBox.appendChild(fehlerHinweis);
        }

        fehlerHinweis.textContent = text;
    }

    // Die Suche wartet auf die JSON-Daten, damit Begriffe nur an einer Stelle gepflegt werden.
```

```
    const glossarDaten = await window.glossarDatenPromise;
    const begriffe = glossarDaten.begriffe;

    if (glossarDaten.ladefehler) {
        searchInput.disabled = true;
        searchInput.placeholder = "Suche derzeit nicht verfügbar";
        zeigeSucheFehler("Glossardaten konnten nicht geladen werden.");
        return;
    }

    // Gefiltert wird nur live nach dem sichtbaren Begriffsnamen.
    searchInput.addEventListener("input", function() {
        const query = this.value.trim().toLocaleLowerCase("de-DE");
        resultsList.innerHTML = "";

        if (query.length === 0) {
            return;
        }

        const filtered = begriffe.filter((begriff) =>
            begriff.name.toLocaleLowerCase("de-DE").includes(query)
        );

        // Treffer werden als normale Links aufgebaut, damit Besuchszustände per CSS sichtbar bleiben.
        filtered.forEach((begriff) => {
            const li = document.createElement("li");
            const link = document.createElement("a");

            link.href = resolveAppPath(begriff.link);
            link.textContent = begriff.name;
            li.appendChild(link);
            resultsList.appendChild(li);
        });
    });
});
```

Anhang 21 - letzteSpeicherung.js

```
/*
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
*/

/*
Zweck:
- Liest `document.lastModified` aus und schreibt den formatierten Wert in den Footer.

Voraussetzung:
- Der Footer wurde bereits aufgebaut und enthält ein Element mit `data-last-modified`.
*/

document.addEventListener("DOMContentLoaded", () => {
    const lastModified = new Date(document.lastModified);
    const lastModifiedElement = document.querySelector("[data-last-modified]");

    if (!lastModifiedElement || Number.isNaN(lastModified.getTime())) {
        return;
    }

    // Das Browser-/Serverdatum wird lokal formatiert und im gemeinsamen Footer angezeigt.
    const formatter = new Intl.DateTimeFormat("de-DE", {
        dateStyle: "long",
        timeStyle: "medium"
    });

    lastModifiedElement.textContent = `Letzte Speicherung: ${formatter.format(lastModified)}`;
});
```

Anhang 22 - footer.js

```
/*
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
*/

/*
Zweck:
- Erzeugt den gemeinsamen Footer zentral auf allen Seiten.

Voraussetzung:
- Die HTML-Seite enthält ein Element `#site-footer`.
- Über `data-base` wird gesteuert, ob die Seite im Root oder im Unterordner `begriffe/` liegt.
*/

document.addEventListener("DOMContentLoaded", () => {
    const footer = document.getElementById("site-footer");

    if (!footer) {
        return;
    }

    const basePath = footer.dataset.base || "";

    // Der Footer wird zentral erzeugt, damit Links und Hinweise nur an einer Stelle gepflegt werden.
    footer.classList.add("page-footer");
    footer.innerHTML = `
        <p class="footer-links">
            <a href="${basePath}index.html">Startseite</a>
            <a href="${basePath}datenschutz.html">Datenschutz</a>
            <a href="${basePath}impressum.html">Impressum</a>
        </p>
        <p class="last-modified" data-last-modified></p>
        <p class="footer-copyright">&copy; 2026 Log7-Consult GmbH. Alle Rechte vorbehalten.</p>
    `;
});
```

Anhang 23 - begriffeDaten.js

```
/*
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
*/

/*
Zweck:
- Lädt die zentrale Datei `src/begriffe.json`.
- Bereinigt und sortiert die Glossarbegriffe für Startseite und Suche.
- Stellt die Daten gebündelt über `window.glossarDaten` und `window.glossarDatenPromise` bereit.

Abhängigkeiten:
- `begriffe.json` muss per Webserver erreichbar sein.
- Die JSON darf nur Links im Format `begriffe/*.html` enthalten.
*/

// Lädt die zentrale JSON-Datei und ergänzt sie um die gemeinsamen Hilfsfunktionen.
const navigation = [
    "A", "Ä", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M",
    "N", "O", "Ö", "P", "Q", "R", "S", "T", "U", "Ü", "V", "W", "X", "Y",
    "Z", "0-9", "#"
];

const glossarCollator = new Intl.Collator("de-DE", {
    sensitivity: "base",
    numeric: true
});

const gruppenReihenfolge = new Map(
    navigation.map((gruppe, index) => [gruppe, index])
);

const datenUrl = new URL("../begriffe.json", document.currentScript.src);
const begriffLinkPattern = /^begriffe\/[^/?#]+\.html$/u;

// Ordnet einen Begriff anhand seines Anfangszeichens einer sichtbaren Navigationsgruppe zu.
function getGlossarGruppe(name) {
    const initial = name.trim().charAt(0).toLocaleUpperCase("de-DE");

    if (!initial) {
        return "#";
    }

    if (/\d/.test(initial)) {
        return "0-9";
    }

    if (["Ä", "Ö", "Ü"].includes(initial)) {
        return initial;
    }
```

```
    if (/[A-Z]/.test(initial)) {
        return initial;
    }

    return "#";
}

// Liefert eine stabile ID für Sprungmarken und Filtergruppen auf der Startseite.
function getGlossarGruppenId(gruppe) {
    const gruppenIds = {
        "Ä": "gruppe-ae",
        "Ö": "gruppe-oe",
        "Ü": "gruppe-ue",
        "0-9": "gruppe-0-9",
        "#": "gruppe-sonderzeichen"
    };

    return gruppenIds[gruppe] || `gruppe-${gruppe.toLowerCase()}`;
}

// Sortiert zuerst nach definierter Gruppierung und innerhalb der Gruppe nach deutscher Alphabetisierung.
function sortiereBegriffe(begriffListe) {
    return [...begriffListe].sort((erstesElement, zweitesElement) => {
        const ersteGruppe = getGlossarGruppe(erstesElement.name);
        const zweiteGruppe = getGlossarGruppe(zweitesElement.name);
        const ersterIndex = gruppenReihenfolge.get(ersteGruppe) ?? navigation.length;
        const zweiterIndex = gruppenReihenfolge.get(zweiteGruppe) ?? navigation.length;

        if (ersterIndex !== zweiterIndex) {
            return ersterIndex - zweiterIndex;
        }

        return glossarCollator.compare(erstesElement.name, zweitesElement.name);
    });
}

// Schützt die Anwendung davor, beliebige oder externe Links aus der JSON zu übernehmen.
function istGueltigerBegriffLink(link) {
    return begriffLinkPattern.test(link);
}

// Aus der JSON werden nur vollständige Einträge übernommen, damit Suche und Startseite stabil bleiben.
function normalisiereBegriffe(begriffListe) {
    const bereinigteBegriffe = [];
    let verworfeneEintraege = 0;

    begriffListe.forEach((begriff) => {
        if (
            !begriff
            || typeof begriff.name !== "string"
            || typeof begriff.link !== "string"
        ) {
            verworfeneEintraege += 1;
```

```
            return;
        }

        const name = begriff.name.trim();
        const link = begriff.link.trim();

        if (!name || !istGueltigerBegriffLink(link)) {
            verworfeneEintraege += 1;
            return;
        }

        bereinigteBegriffe.push({
            name,
            link,
            beschreibung: typeof begriff.beschreibung === "string"
                ? begriff.beschreibung.trim()
                : ""
        });
    });

    if (verworfeneEintraege > 0) {
        console.warn(
            `${verworfeneEintraege} Glossareintrag/Glossareinträge wurden verworfen, weil Pflichtfelder fehlten oder der Link nicht dem
Muster "begriffe/*.html" entsprach.`
        );
    }

    return bereinigteBegriffe;
}

// Bündelt die Rohdaten mit allen Hilfsfunktionen in einem Objekt für die übrigen Skripte.
function erstelleGlossarDaten(begriffListe, ladefehler = false) {
    return {
        navigation,
        begriffe: sortiereBegriffe(normalisiereBegriffe(begriffListe)),
        getGruppe: getGlossarGruppe,
        getGruppenId: getGlossarGruppenId,
        sortiereBegriffe,
        ladefehler
    };
}

// Lädt die Glossarbegriffe einmalig und erstellt daraus das gemeinsame Datenobjekt.
async function ladeGlossarDaten() {
    const response = await fetch(datenUrl);

    if (!response.ok) {
        throw new Error(`Glossardaten konnten nicht geladen werden: ${response.status}`);
    }

    const rohdaten = await response.json();
    const begriffe = Array.isArray(rohdaten.begriffe) ? rohdaten.begriffe : [];

    return erstelleGlossarDaten(begriffe);
```

```
}

// Andere Skripte warten auf dieses Promise, damit die JSON nur einmal geladen wird.
window.glossarDatenPromise = ladeGlossarDaten()
    .then((daten) => {
        window.glossarDaten = daten;
        return daten;
    })
    .catch((error) => {
        console.error("Fehler beim Laden der Glossardaten.", error);

        const fallbackDaten = erstelleGlossarDaten([], true);
        window.glossarDaten = fallbackDaten;
        return fallbackDaten;
    });
```

Anhang 24 - logo.js

```
/*
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
*/

/*
Zweck:
- Baut die gemeinsame Topbar mit Logo und Suche auf.

Voraussetzung:
- Die HTML-Seite enthält ein Element `#site-topbar`.
- Über `data-base` wird das passende Pfadpräfix für Root- und Unterordnerseiten gesetzt.
*/

document.addEventListener("DOMContentLoaded", () => {
    const topbarHost = document.getElementById("site-topbar");

    if (!topbarHost) {
        return;
    }

    const basePath = topbarHost.dataset.base || "";

    // Logo und Suche werden gemeinsam erzeugt, damit der obere Seitenbereich nur einmal gepflegt wird.
    topbarHost.classList.add("site-topbar");
    topbarHost.innerHTML = `
        <a class="site-logo" href="${basePath}index.html" aria-label="Zur Startseite">
            <img class="site-logo-image" src="${basePath}logo.png" alt="Log7-Consult">
        </a>
        <div class="search-box">
            <input type="text" id="search" placeholder="Begriff suchen..." autocomplete="off">
            <ul id="results"></ul>
        </div>
    `;
});
```

Anhang 25 - startseiteGlossar.js

```
/*
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
*/

/*
Zweck:
- Rendert auf der Startseite die A-Z-Filterleiste und die tabellarische Glossarübersicht.
- Nutzt dafür die bereits geladenen JSON-Daten und die gemeinsame Vorlesefunktion.

Abhängigkeiten:
- `window.glossarDatenPromise` aus `begriffeDaten.js`
- `window.vorlesenHelfer` aus `vorlesen.js`
*/

document.addEventListener("DOMContentLoaded", async () => {
    const alphabetNavigation = document.getElementById("alphabet-nav");
    const glossaryTableBody = document.getElementById("glossary-table-body");

    if (!alphabetNavigation || !glossaryTableBody || !window.glossarDatenPromise || !window.vorlesenHelfer) {
        return;
    }

    const glossarDaten = await window.glossarDatenPromise;
    const { navigation, begriffe, getGruppe, getGruppenId, sortiereBegriffe } = glossarDaten;
    const sortierteBegriffe = sortiereBegriffe(begriffe);
    const aktiveGruppen = new Set(sortierteBegriffe.map((begriff) => getGruppe(begriff.name)));
    const ausgewaehlteGruppen = new Set();
    const { kannVorlesen, erstelleVorlesenButton } = window.vorlesenHelfer;

    // Zeigt Rückmeldungen direkt im Tabellenbereich an, ohne ein separates Hinweisfeld zu benötigen.
    function renderHinweis(text) {
        glossaryTableBody.innerHTML = "";

        const zeile = document.createElement("tr");
        const zelle = document.createElement("td");

        zelle.textContent = text;
        zeile.appendChild(zelle);
        glossaryTableBody.appendChild(zeile);
    }

    if (glossarDaten.ladefehler) {
        renderHinweis("Glossardaten konnten nicht geladen werden.");
        return;
    }

    // Baut die Tabelle jedes Mal neu auf, damit Mehrfachfilter konsistent angewendet werden.
    function renderTabelle() {
        const sichtbareBegriffe = sortierteBegriffe.filter((begriff) => {
```

```
    if (ausgewaehlteGruppen.size === 0) {
        return true;
    }

    return ausgewaehlteGruppen.has(getGruppe(begriff.name));
});

let aktuelleGruppe = "";
glossaryTableBody.innerHTML = "";

if (sichtbareBegriffe.length === 0) {
    renderHinweis("Keine Begriffe für die aktuelle Auswahl vorhanden.");
    return;
}

sichtbareBegriffe.forEach((begriff) => {
    const gruppe = getGruppe(begriff.name);

    if (gruppe !== aktuelleGruppe) {
        const gruppenZeile = document.createElement("tr");
        const gruppenZelle = document.createElement("th");

        gruppenZeile.id = getGruppenId(gruppe);
        gruppenZeile.className = "glossary-letter-row";
        gruppenZelle.colSpan = 1;
        gruppenZelle.textContent = gruppe;
        gruppenZeile.appendChild(gruppenZelle);
        glossaryTableBody.appendChild(gruppenZeile);

        aktuelleGruppe = gruppe;
    }

    // Jede Tabellenzeile enthält den Link zum Begriff und den Vorlese-Button.
    const zeile = document.createElement("tr");
    const begriffsZelle = document.createElement("th");
    const begriffsInhalt = document.createElement("div");
    const link = document.createElement("a");
    const vorlesenButton = erstelleVorlesenButton({
        text: begriff.name,
        ariaLabel: `${begriff.name} vorlesen`
    });

    begriffsZelle.scope = "row";
    begriffsInhalt.className = "glossary-entry";
    link.href = begriff.link;
    link.textContent = begriff.name;
    vorlesenButton.disabled = !kannVorlesen;

    begriffsInhalt.appendChild(link);
    begriffsInhalt.appendChild(vorlesenButton);
    begriffsZelle.appendChild(begriffsInhalt);

    zeile.appendChild(begriffsZelle);
    glossaryTableBody.appendChild(zeile);
```

```
        });
    }

    navigation.forEach((gruppe) => {
        const element = document.createElement(aktiveGruppen.has(gruppe) ? "button" : "span");

        element.textContent = gruppe;
        element.className = aktiveGruppen.has(gruppe) ? "alphabet-link" : "alphabet-label";

        if (aktiveGruppen.has(gruppe)) {
            element.type = "button";
            element.setAttribute("aria-pressed", "false");

            // Mehrfachauswahl bleibt aktiv, bis ein Buchstabe erneut angeklickt wird.
            element.addEventListener("click", () => {
                if (ausgewaehlteGruppen.has(gruppe)) {
                    ausgewaehlteGruppen.delete(gruppe);
                    element.classList.remove("is-active");
                    element.setAttribute("aria-pressed", "false");
                } else {
                    ausgewaehlteGruppen.add(gruppe);
                    element.classList.add("is-active");
                    element.setAttribute("aria-pressed", "true");
                }

                renderTabelle();
            });
        } else {
            element.setAttribute("aria-disabled", "true");
        }

        alphabetNavigation.appendChild(element);
    });

    renderTabelle();
});
```

Anhang 26 - vorlesen.js

```
/*
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
*/

/*
Zweck:
- Stellt die gemeinsame Vorlesefunktion auf Basis der Browser-Sprachausgabe bereit.
- Ergänzt Vorlese-Buttons auf der Startseite und auf den Begriffsseiten.

Bereitgestellte Schnittstelle:
- `window.vorlesenHelfer.kannVorlesen`
- `window.vorlesenHelfer.liesVor(text)`
- `window.vorlesenHelfer.erstelleVorlesenButton(optionen)`
*/

(function() {
    const kannVorlesen = "speechSynthesis" in window && "SpeechSynthesisUtterance" in window;

    // Entfernt unnötige Zeilenumbrüche, damit die Sprachausgabe natürlicher klingt.
    function bereinigeText(text) {
        return text.replace(/\s+/g, " ").trim();
    }

    // Vor jeder neuen Ausgabe wird die laufende Sprachausgabe gestoppt.
    function liesVor(text) {
        if (!kannVorlesen) {
            return;
        }

        window.speechSynthesis.cancel();

        const ausgabe = new SpeechSynthesisUtterance(bereinigeText(text));
        ausgabe.lang = "de-DE";
        ausgabe.rate = 0.95;

        window.speechSynthesis.speak(ausgabe);
    }

    // Der Button-Aufbau ist zentralisiert, damit Startseite und Begriffsseiten identisch bleiben.
    function erstelleVorlesenButton(optionen) {
        const button = document.createElement("button");

        button.type = "button";
        button.className = optionen.className || "read-aloud-button";
        button.innerHTML = "&#128266;";
        button.setAttribute("aria-label", optionen.ariaLabel);
        button.title = optionen.title || optionen.ariaLabel;
        button.disabled = !kannVorlesen;
```

```
        button.addEventListener("click", () => {
            liesVor(optionen.text);
        });

        return button;
    }

    document.addEventListener("DOMContentLoaded", () => {
        const titleGroup = document.querySelector(".title-group");
        const title = titleGroup ? titleGroup.querySelector("h1") : null;
        const begriffName = title ? bereinigeText(title.textContent) : "";

        // Auf Begriffsseiten wird der Seitentitel separat vorlesbar gemacht.
        if (titleGroup && title && !titleGroup.querySelector(".title-read-button")) {
            title.insertAdjacentElement("afterend", erstelleVorlesenButton({
                text: begriffName,
                ariaLabel: `${begriffName} vorlesen`,
                className: "read-aloud-button title-read-button"
            }));
        }

        document.querySelectorAll(".highlight").forEach((definition) => {
            const definitionText = bereinigeText(definition.textContent);
            const definitionHeading = definition.previousElementSibling;

            // Der Definitions-Button sitzt an der H2, damit die Aktion sofort erkennbar bleibt.
            if (definitionHeading && definitionHeading.matches("h2")) {
                if (definitionHeading.querySelector(".definition-read-button")) {
                    return;
                }

                definitionHeading.classList.add("definition-heading");
                definitionHeading.appendChild(erstelleVorlesenButton({
                    text: `Definition von ${begriffName}. ${definitionText}`,
                    ariaLabel: `Definition von ${begriffName} vorlesen`,
                    className: "read-aloud-button definition-read-button"
                }));
            }
        });
    });

    // Diese globale Hilfsschnittstelle wird von `startseiteGlossar.js` wiederverwendet.
    window.vorlesenHelfer = {
        kannVorlesen,
        liesVor,
        erstelleVorlesenButton
    };
})();
```

Anhang 27 - zurueckLink.js

```
/*
Autor: Christopher Oelschläger
Erstelldatum: 29.03.2026
Änderungshistorie:
- Version 1.0: Initiale Version
*/

/*
Zweck:
- Verleiht dem Zurück-Pfeil auf Begriffsseiten ein browserbasiertes Zurück-Verhalten.

Fallback:
- Wenn kein sinnvoller Verlauf vorhanden ist, bleibt der normale Link zur Startseite aktiv.
*/

document.addEventListener("DOMContentLoaded", () => {
    const backLink = document.querySelector("[data-back-link]");

    if (!backLink) {
        return;
    }

    // Mit Verlauf geht es wirklich zurück, ohne Verlauf greift einfach der normale Link zur Startseite.
    backLink.addEventListener("click", (event) => {
        if (document.referrer || window.history.length > 1) {
            event.preventDefault();
            window.history.back();
        }
    });
});
```

Anhang 28 - begriffe.json


```
{
  "dokumentenkopf": {
    "autor": "Christopher Oelschläger",
    "erstelldatum": "29.03.2026",
    "aenderungshistorie": [
      {
        "version": "1.0",
        "beschreibung": "Initiale Version"
      }
    ]
  },
  "begriffe": [
    {
      "name": "Äquivalenzklassenbildung",
      "link": "begriffe/aequivalenzklassenbildung.html",
      "beschreibung": "Testtechnik zur Reduktion von Testfällen durch Gruppierung ähnlicher Eingaben."
    },
    {
      "name": "Testabdeckung",
      "link": "begriffe/testabdeckung.html",
      "beschreibung": "Maß dafür, wie gut Testfälle die definierten Bedingungen abdecken."
    },
    {
      "name": "Testbasis",
      "link": "begriffe/testbasis.html",
      "beschreibung": "Dokumente und Informationen, aus denen Testbedingungen abgeleitet werden."
    },
    {
      "name": "Testbedingung",
      "link": "begriffe/testbedingung.html",
      "beschreibung": "Ein prüfbares Element, das durch Testfälle validiert wird."
    },
    {
      "name": "Testfall",
      "link": "begriffe/testfall.html",
      "beschreibung": "Beschreibung konkreter Eingaben, Aktionen und erwarteter Ergebnisse zur Überprüfung."
    }
  ]
}
```

Wartungsanleitung

Diese Anleitung ist für Mitarbeitende gedacht, die Inhalte pflegen möchten, ohne tief in HTML, CSS oder JavaScript einzusteigen.

Das Wichtigste vorab:

- Für neue oder geänderte Begriffe müssen in der Regel nur zwei Stellen bearbeitet werden.
- Sie müssen normalerweise **kein** JavaScript und **kein** CSS ändern.
- Arbeiten Sie langsam und speichern Sie nach jedem Schritt.

Was Sie für die Pflege brauchen

- Einen Texteditor, zum Beispiel Notepad.
- Zugriff auf den Projektordner.
- Etwas Sorgfalt beim Schreiben von Dateinamen.

Welche Dateien wichtig sind

Für die Inhaltspflege sind fast immer nur diese Dateien wichtig:

- `src/begriffe/_vorlage.html`
- `src/begriffe.json`
- die einzelne Begriffsdatei im Ordner `src/begriffe/` , zum Beispiel `src/begriffe/testfall.html`

Diese Dateien sollten Sie für normale Inhaltspflege **nicht** ändern:

- `src/js/*.js`
- `src/style.css`

Wann Sie was bearbeiten müssen

Wenn Sie nur den Text eines vorhandenen Begriffs ändern wollen:

1. Öffnen Sie die passende HTML-Datei im Ordner `src/begriffe/` .
2. Passen Sie den Text an.